



Institut Bernat el Ferrer



Institut Bernat el Ferrer

DESENVOLUPAMENT D'APLICACIONS WEB

Segon Curs

WHATIZIT

Autor: Miquel Manzano & Joao Meneses

Data d'entrega:

Curs acadèmic 2025/2026

Resum

En aquest apartat es fa un resum del document i del projecte per situar al lector que llegirà a continuació (no més de deu línies), pot començar:

Este documento presenta el resultado del proyecto de desarrollo de una aplicación web interactiva basada en un sistema de juego en tiempo real. La aplicación permite a los usuarios registrarse, acceder a diferentes salas organizadas por categorías y participar en partidas donde deben poner a prueba sus conocimientos. A lo largo del proyecto se han implementado funcionalidades como la gestión de usuarios, categorías, salas y partidas, así como un sistema de puntuación y ranking. Además, se ha desarrollado un panel de administración para gestionar los recursos de la aplicación. El sistema se ha construido utilizando tecnologías como Laravel en el backend y Vue 3 en el frontend, siguiendo una arquitectura cliente-servidor.

Paraules clau

Paraules clau sobre el projecte separades per comes, exemple:

Aplicación web, juego en tiempo real, Laravel, Vue 3, API REST, salas, categorías, ranking, usuarios, base de datos

Keywords

El mateix que l'anterior, però aquesta vegada en anglés.

Web application, real-time game, Laravel, Vue 3, REST API, rooms, categories, ranking, users, database

Índex

- 1 Introducció
- 2 Context i motivació del projecte
 - 2.1 Objectius del projecte
 - 2.1.1 Objectius generals de producció
 - 2.1.2 Objectius d'aprenentatge
- 3 Descripció
 - 3.1 Resultats esperats
 - 3.2 Beneficis
- 4 Anàlisi
 - 4.1 Requisits funcionals
 - 4.2 Requisits no funcionals
 - 4.3 Requisits de dades
- 5 Disseny
 - 5.1 Definició de l'empresa original
 - 5.2 Definició de l'empresa de restauració e identificació del target
 - 5.1 Identificació de la imatge corporativa de l'empresa
 - 5.1.1 Guia d'estil per la implementació de disseny (moodboard)
 - 5.1 Disseny de la interfície
 - 5.2 Flux de l'aplicació
 - 5.3 Disseny del Sistema
 - 5.4 Tests d'usabilitat i accessibilitat (anàlisi heurístic)
- 6 Resultat Final
- 7 Conclusions
- 8 Referències

1 Introducció

2 Context i motivació del projecte

2.1 Objectius del projecte

2.1.1 Objectius generals de producció

- Desarrollar una aplicación web completa utilizando Laravel en el backend y Vue 3 en el frontend, siguiendo una arquitectura cliente-servidor.
- Implementar un sistema de juego interactivo basado en salas y categorías, donde los usuarios puedan competir entre sí.
- Diseñar y estructurar una base de datos eficiente que permita gestionar correctamente las relaciones entre entidades como usuarios, partidas, salas y categorías.
- Aplicar buenas prácticas de programación, manteniendo un código organizado, reutilizable y coherente en todo el proyecto.
- Implementar un sistema de autenticación y control de acceso mediante roles y permisos, garantizando la seguridad de la aplicación.
- Utilizar Axios para gestionar las peticiones HTTP desde el frontend.
- Crear un panel de administración que permita gestionar todos los recursos de la aplicación.
- Implementar funcionalidades de filtrado, búsqueda y paginación tanto en backend como en frontend.
- Diseñar una interfaz de usuario atractiva, intuitiva y responsive utilizando Bootstrap.
- Garantizar el correcto funcionamiento de la aplicación mediante pruebas básicas de los endpoints principales.
- Utilizar Git como herramienta de control de versiones para el trabajo en equipo.

2.1.2 Objectius d'aprenentatge

Amb aquest projecte es busca adquirir, repassar i practicar els següents conceptes:

- Aprendre el llenguatge de programació PHP.
- Aprendre l'aplicació de conceptes de disseny web.
- Creació d'interfícies web
- Aprendre a analitzar e identificar errors d'UX/UI.
- Aprendre a utilitzar els diferents elements d'HTML i fulls d'estil.
- Implantació de Bootstrap.
- Combinació de PHP amb HTML.
- Creació de codi net i estructurat.
- Creació i utilització de classes PHP.
- Aprendre, comprendre i portar a pràctica els conceptes d'abstracció polimorfisme i classes abstractes.
- Ús i administració de sessions amb PHP.
- Treballar l'ús avançat de cookies.

3 Resultats esperats

Escriu de forma general, sense entrar en detall quin resultat sobre el projecte una vegada realitzat el projecte.

El proyecto consiste en el desarrollo de una aplicación web interactiva basada en un juego de adivinanza en tiempo real, donde los usuarios pueden poner a prueba sus conocimientos en diferentes categorías.

Una vez finalizado, el sistema permitirá a los usuarios registrarse, acceder a salas de juego y participar en partidas donde deberán identificar elementos (como imágenes o conceptos) en el menor tiempo posible. En función de su rendimiento, los jugadores obtendrán puntuaciones que se reflejarán en rankings globales y por categorías.

La aplicación contará con un sistema de categorías que permitirá organizar las partidas según diferentes temáticas (por ejemplo: deportes, cine, música, etc.), así como salas en las que los usuarios podrán competir. Además, incluirá un sistema de ranking que mostrará la posición de los jugadores en función de sus puntuaciones.

Desde el punto de vista del usuario, la aplicación permitirá:

- Registrarse e iniciar sesión.
- Acceder a diferentes salas de juego.
- Participar en partidas en tiempo real. (opcional)
- Acumular puntos según su rendimiento.
- Consultar rankings globales y por categorías.

Por otro lado, el sistema contará con un panel de administración desde el cual se podrán gestionar los distintos recursos de la aplicación (usuarios, categorías, salas, etc.), incluyendo la creación, edición y eliminación de los mismos, así como la gestión de permisos y roles.

4 Anàlisi

En el següent apartat es mostrarà l'anàlisi realitzat pel desenvolupament del projecte. En ell es recullen els requisits del sistema, requisits de dades i els requisits no funcionals.

4.1 Requisits funcionals

Explicar totes les funcions (requisits) que tindrà l'aplicació.

- **RF00:** La aplicación permitirá el acceso a una página principal donde se mostrará información general del juego y un botón para comenzar a jugar.
- **RF01:** La aplicación permitirá a los usuarios registrarse y autenticarse mediante un sistema de login con email y contraseña.
- **RF02:** El sistema permitirá diferenciar usuarios normales y usuarios administradores mediante un sistema de roles y permisos.

Gestion de salas:

- **RF04:** El sistema permitirá crear salas de juego en modo individual.
- **RF05:** Cada sala contará con un nombre, un código identificativo y un usuario creador.
- **RF06:** El sistema permitirá asociar una o varias categorías a cada sala.
- **RF09:** El sistema permitirá visualizar un listado de salas disponibles.
- **RF10:** El sistema permitirá acceder a una sala introduciendo su código o seleccionando desde el listado.
- **RF11:** El sistema permitirá editar los datos de una sala (nombre y categorías asociadas) por parte de su creador o un administrador
- **RF12:** El sistema permitirá eliminar una sala por parte de su creador o un administrador.

Gestion de categorías:

- **RF13:** El sistema permitirá crear categorías para clasificar las imágenes del juego.
- **RF14:** Cada categoría tendrá un nombre único y descriptivo.
- **RF15:** El sistema permitirá visualizar un listado de categorías disponibles.

Desarrollo del juego (modo individual)

- **RF16:** El sistema permitirá iniciar una partida individual dentro de una sala seleccionada.
- **RF17:** Durante la partida se mostrará una imagen al usuario para que intente adivinar su contenido.
- **RF18:** El usuario podrá introducir una respuesta mediante un campo de texto.
- **RF20:** El sistema validará automáticamente si la respuesta introducida es correcta o incorrecta.
- **RF21:** El sistema asignará una puntuación al usuario en función de si la respuesta es correcta y del tiempo empleado en responder.
- **RF22:** El sistema registrará cada respuesta del usuario asociándose a la imagen y a la partida correspondiente.

Gestion de imágenes:

- **RF23:** El sistema permitirá al administrador añadir imágenes al juego indicando la respuesta correcta asociada.
- **RF24:** El sistema permitirá asociar una imagen a una o varias categorías.
- **RF25:** El sistema permitirá mostrar imágenes aleatorias filtradas por categoría dentro de una sala.

Ranking y puntuaciones:

- **RF26:** El sistema almacenará la puntuación total obtenida por cada usuario en las partidas jugadas.
- **RF27:** El sistema permitirá visualizar un ranking global con los usuarios mejor clasificados.
- **RF28::** El ranking se reiniciará periódicamente para fomentar la participación de nuevos usuarios.

Panel de administración:

- **RF29:** El sistema dispondrá de un panel de administración accesible únicamente para usuarios con rol administrador.
- **RF30:** Desde el panel de administración se podrán gestionar usuarios, salas, categorías e imágenes.

Funcionalidades futuras:



- **RF31:***El sistema podrá ampliarse para permitir partidas multijugador en tiempo real dentro de una misma sala.*
- **RF32:***En el modo multijugador, los usuarios competirán entre sí por obtener la mayor puntuación.*

Un requisit funcional no pot englobar més d'una funció, els exemples anteriors són RF separats, no poden anar junts.

4.2 Requisits no funcionals

Els requisits no funcionals són aquells que imposen restriccions sota les quals ha de funcionar el sistema.

En el nostre sistema tenim els següents requisits no funcionals:

- *RNF01 Fluïdesa: Malgrat tractar amb moltes imatges i dependre de la connexió a Internet, és molt important que sigui una aplicació amb una resposta ràpida.*

RNF02 Usabilitat: L'aplicació ha de ser intuïtiva perquè es pugui ser usada per qualsevol tipus d'usuari.

Gestion de salas:

RNF01: El sistema deberá permitir la creación y acceso a salas de juego con un tiempo de respuesta rápido, evitando esperas prolongadas para el usuario.

RNF02: La interfaz de gestión de salas deberá ser clara e intuitiva, permitiendo a los usuarios crear y unirse a salas de forma sencilla.

RNF03: El listado de salas disponibles deberá estar accesible siempre que el sistema se encuentre operativo.

RNF04: El sistema deberá garantizar que solo el creador de la sala o un usuario administrador pueda modificar o eliminar una sala.

RNF05: Cada sala deberá disponer de un código identificativo único y mantener correctamente asociados su nombre, categorías y usuario creador.

RNF06: El sistema deberá ser capaz de gestionar un número creciente de salas creadas por los usuarios sin afectar al rendimiento.

RNF07: El acceso a una sala mediante código o selección desde el listado deberá realizarse sin errores ni accesos incorrectos.

RNF08: Los cambios realizados en una sala (edición o eliminación) deberán reflejarse inmediatamente en el sistema.

Gestion de categorías:

RNF01: El sistema deberá permitir la creación de categorías con un tiempo de respuesta rápido.

RNF02: La interfaz de gestión de categorías deberá ser clara e intuitiva para facilitar su uso.

RNF03: El listado de categorías disponibles deberá mostrarse correctamente siempre que el sistema esté operativo.

Desarrollo del juego (modo individual):

RNF01: El sistema deberá permitir iniciar una partida individual con un tiempo de respuesta rápido.

RNF02: La visualización de las imágenes durante la partida deberá realizarse de forma fluida y sin interrupciones.

RNF03: La introducción de respuestas por parte del usuario deberá ser sencilla y sin errores de funcionamiento.

RNF04: La validación de las respuestas deberá realizarse de forma automática e inmediata.

RNF05: El cálculo y asignación de la puntuación deberá realizarse de manera precisa y coherente.

RNF06: El registro de las respuestas del usuario deberá realizarse correctamente sin pérdida de información.

Gestion de imagenes:

RNF01: La carga de imágenes al sistema deberá realizarse de forma rápida y sin errores.

RNF02: Las imágenes deberán mantenerse correctamente asociadas a sus categorías correspondientes.

RNF03: La selección de imágenes aleatorias filtradas por categoría deberá realizarse de forma eficiente.

Ranking y puntuaciones:

RNF01: El sistema deberá almacenar las puntuaciones de los usuarios de forma segura y confiable.

RNF02: La visualización de los rankings deberá ser rápida y consistente, sin retrasos en la actualización.

RNF03: El reinicio periódico de los rankings deberá realizarse automáticamente sin afectar la disponibilidad del sistema.

Panel de administración:

RNF01: El panel de administración deberá ser accesible únicamente por usuarios con rol de administrador.

RNF02: La gestión de usuarios, salas, categorías e imágenes desde el panel deberá realizarse de forma rápida y segura.

Funcionalidades futuras:

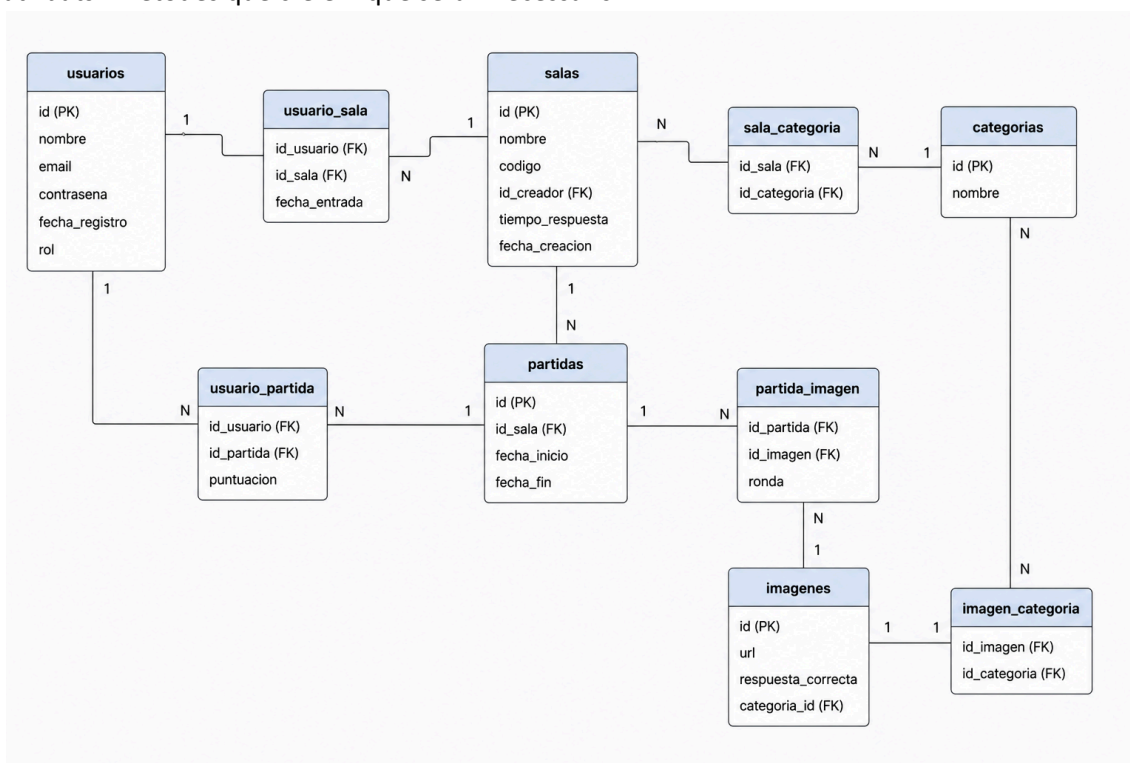
RNF01: El sistema deberá estar diseñado de manera modular y escalable para permitir la ampliación a partidas multijugador en tiempo real sin afectar el rendimiento de las funcionalidades existentes.

RNF02: La interacción y competencia entre usuarios en el modo multijugador deberá realizarse de forma fiable y segura, asegurando la correcta asignación de puntuaciones y evitando conflictos de datos en tiempo real.

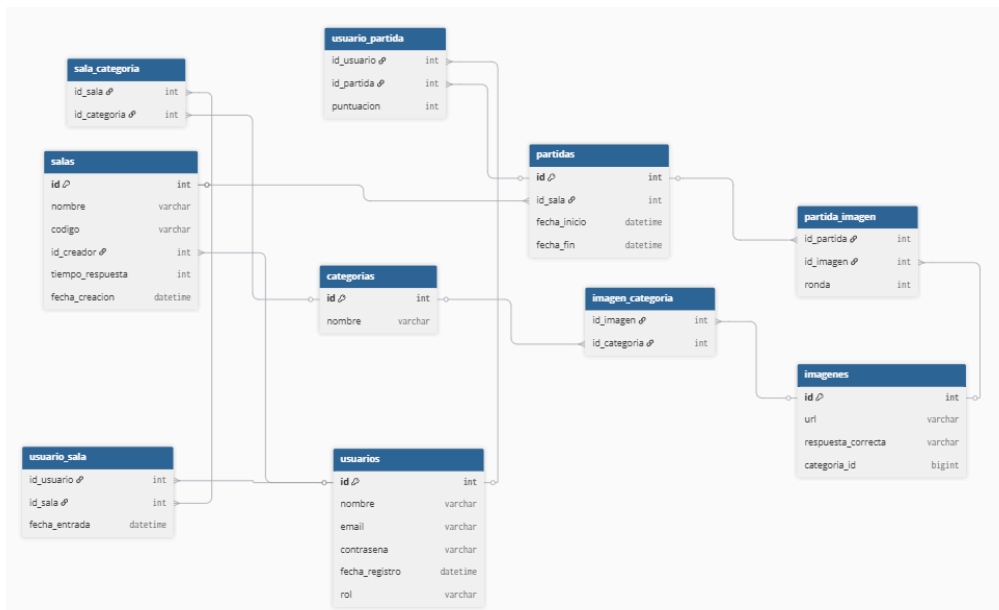
5 Requisits de dades

5.1 Diagrama de classes

Afegirem el diagrama de classes, les classes que posteriorment crearem, els seus atributs i mètodes que creiem que seran necessaris.



5.2 Disseny de base de dades



Relación	Tipo
Usuarios – Salas	N-M
Usuarios – Partidas	N-M
Partidas – Imágenes	N-M
Usuarios – Imágenes	N-M
Salas – Categorías	N-M
Imágenes – Categorías	N-M

5.3 Disseny del sistema

Esquema del model client-servidor amb les tecnologies.



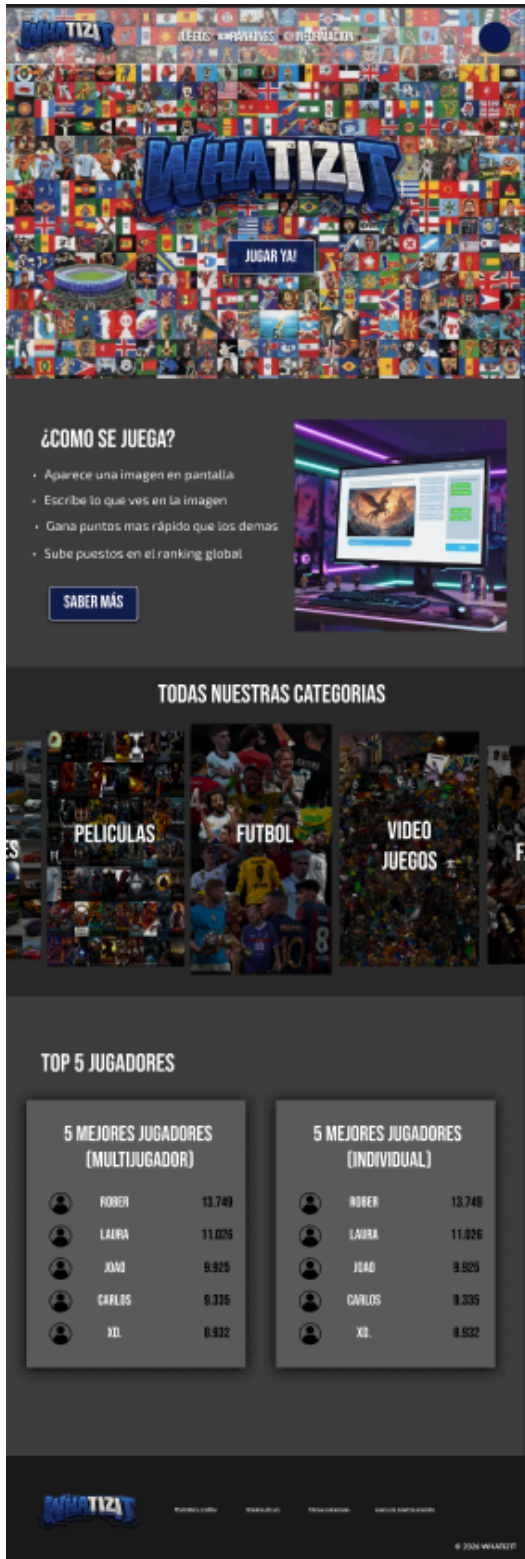
El esquema del sistema describe un modelo Cliente-Servidor estructurado en tres niveles principales. El primer nivel es el Cliente, que corresponde al navegador del usuario y está construido con Vue 3 utilizando Composition API, Bootstrap para la interfaz gráfica y Axios para las peticiones HTTP. Este cliente interactúa con la siguiente capa mediante peticiones HTTP basadas en una API JSON, empleando los métodos GET, POST, PUT y DELETE. El segundo nivel es el Servidor o backend, desarrollado con Laravel API, donde se gestiona la lógica del sistema a través de controladores, rutas REST, middleware para la autenticación, policies para los permisos y procesos de validaciones. Este servidor se conecta con el tercer y último nivel utilizando el ORM Eloquent. El nivel final es la Base de Datos gestionada con MySQL, la cual almacena la información en distintas tablas principales, tales como usuarios, salas, categorías, partidas, imagenes, sala_categoria e imagen_categoria, etc.

6 Disseny

Aquesta secció ha d'incloure la informació de:

- Creació d'un **prototip** de disseny per a escriptori.

Este era nuestro diseño principal, pero teníamos muchas dudas sobre los colores y estructura y decidimos crear otro diseño.



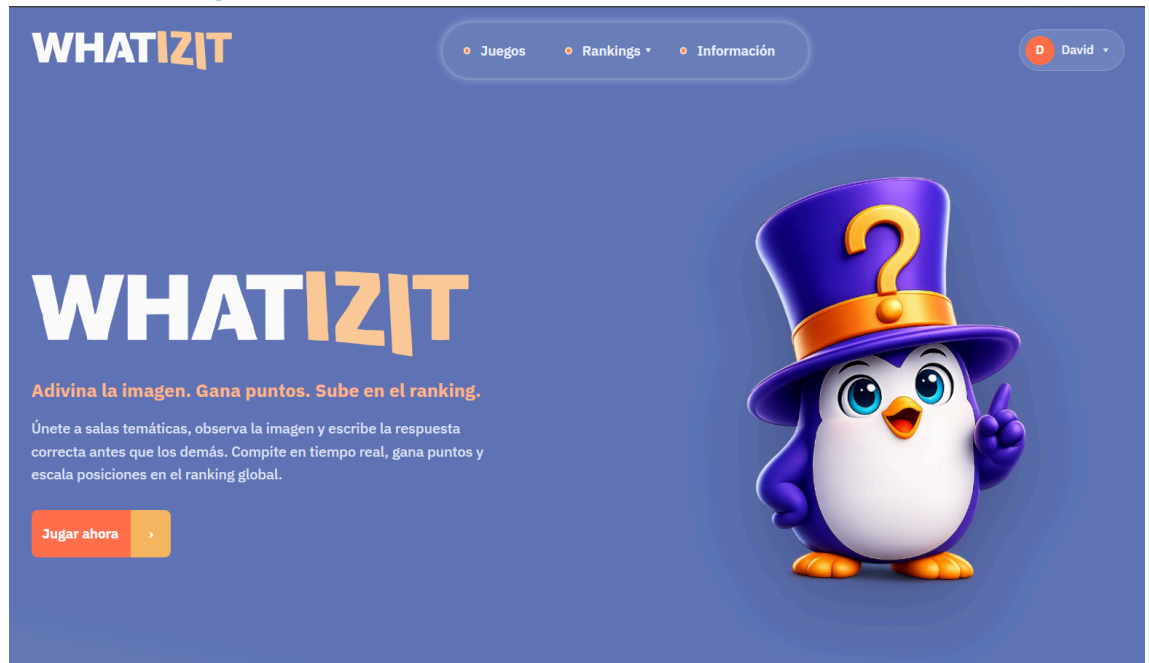
- **Correcció del prototip** de disseny.

Con ayuda de Laura nos ayudó a encontrar una referencia para inspirarnos y nos inspiramos en esta web.Y la empezamos a desarrollar a nuestro estilo.

<https://dribbble.com/shots/26546011-game-stream-landing-page-web-ui-design>

n

- Creació **pàgina home** amb header, 3 o 4 seccions de informació i footer.
Está la sección del header y el navegador donde tenemos el logo, el menu de las secciones y el perfil o el login y registre si no estás registrado. En el header tenemos una descripción de qué trata nuestro juego y un botón donde te redirige a jugar. Y tenemos una imagen de la mascota de la web.



La segunda sección es una sección de información de cómo jugar al juego y un pequeño resumen de cómo funciona el juego y después tenemos unas características que definen nuestro juego





Después tenemos un pequeño ranking global donde salen los jugadores que han jugado en nuestras salas. Solo sala el top 5 del ranking en individual ya que en multijugador no nos ha dado tiempo pero lo hemos dejado.

WHATIZIT

[Juegos](#) [Rankings](#) [Información](#)

David

Mejores Jugadores

Individual Multijugador

JUGADOR	ELO	P. JUGADAS	TÍTULO
1. JOAO	0	0	BRONCE
2. DAVID	0	0	BRONCE
3. USER	0	0	BRONCE
4. DAVID	0	0	BRONCE

Y finalmente tenemos una sección donde salen las categorías predeterminadas que tenemos creadas para que puedas jugar en esa sala. Y como último el footer donde tenemos el logo y unos apartados y las redes sociales

Categorías

ANIMALES

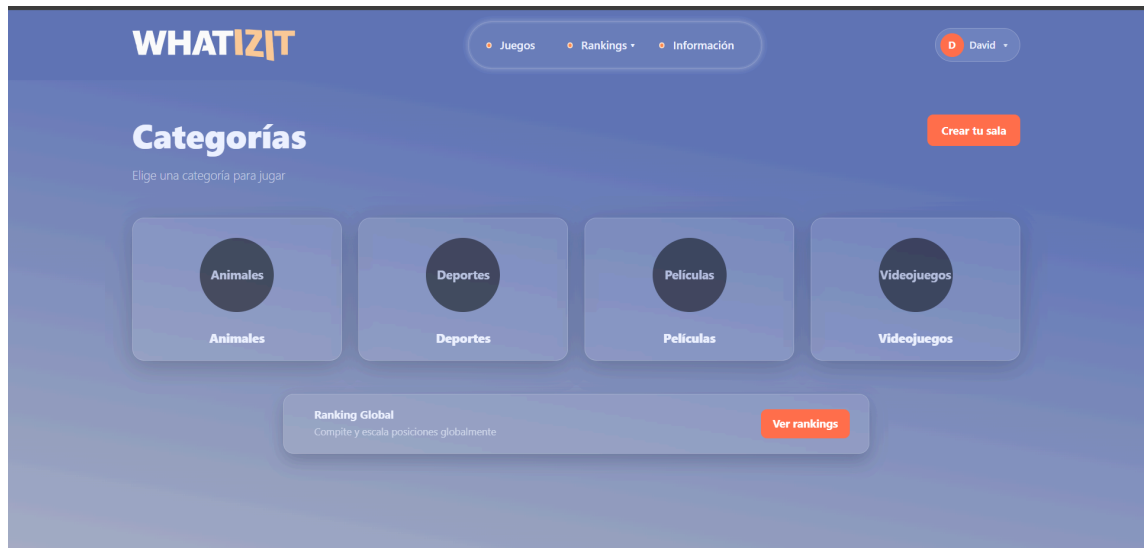
ANIMALES

¡Jugar ahora!

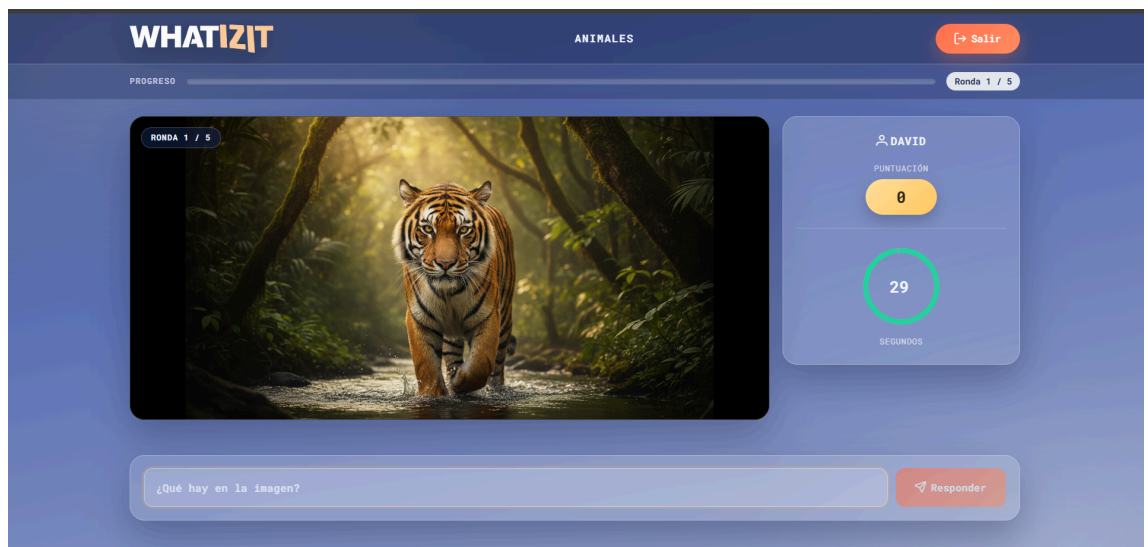
WHATIZIT

[Funciones](#) [Rankings](#) [FAQ](#) [Contacto](#)

- Creació **Pàgina productes o carta** amb tots els productes que es poden afegir.
Página de las salas por categorías predeterminadas donde podrás jugar en esas ciertas salas. Después tenemos el botón para crear tus propias salas y mezclar categorías, y un botón para ir al ranking.



Después esta sería la pantalla de juego donde salen las imágenes un temporizador y para poder escribir la respuesta



- Creació **Pàgina finalitzar compra** amb els productes afegits i opció de finalitzar compra amb preu i impostos.

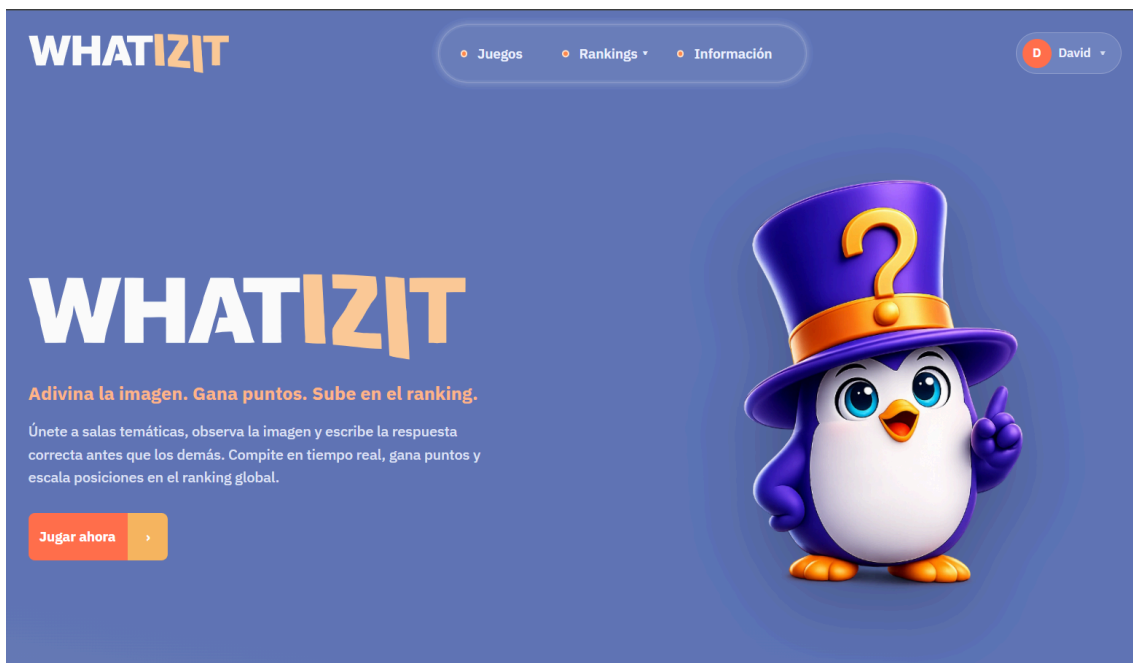
Y cuando finalizas el juego se ve esta pantalla con tu puntuación que has hecho en la partida.



7 Resultat Final

Resultat final de la web amb captures.

Página de inicio:



CÓMO FUNCIONA

¿CÓMO SE JUEGA?

1. Entra en una sala de cualquier categoría
2. Aparece una imagen en pantalla
3. Escribe lo que crees que es antes que nadie
4. Gana puntos y sube en el ranking global

[Ver cómo jugar](#)

COMPETICIÓN

Pon a prueba tus conocimientos

Compite en tiempo real contra jugadores de todo el mundo. Cuanto más rápido aciertes, más puntos consigues. Cada partida cuenta para tu posición en el ranking.

[Competir ahora](#)

Categorías variadas



Ranking global



Responde rápido



Salas en tiempo real

Mejores Jugadores

Individual

Multijugador

JUGADOR	ELO	P. JUGADAS	TÍTULO
1.  DAVID	1780	0	SILVER
2.  USER	1300	10	ORO
3.  DAVID	610	0	BRONZE
4.  GOTI	0	0	BRONCE

Categorías

ANIMALES

ANIMALES

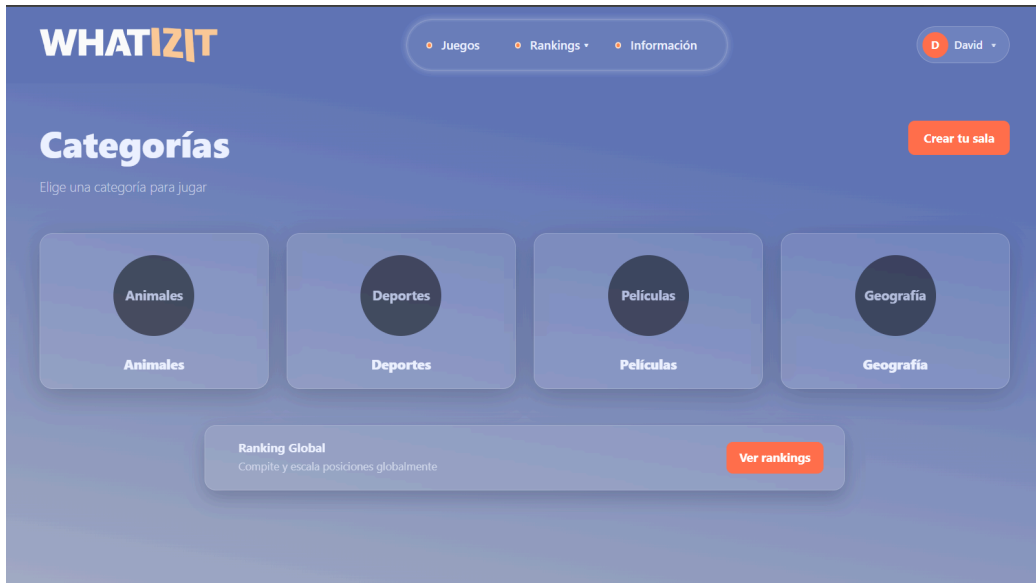
[¡Jugar ahora!](#)

WHATIZIT

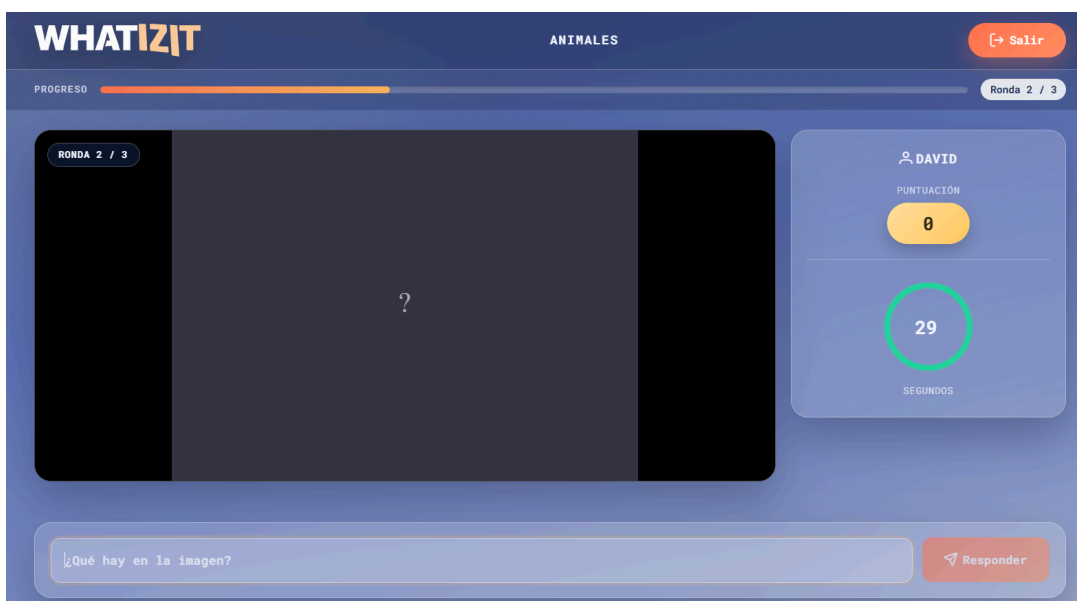
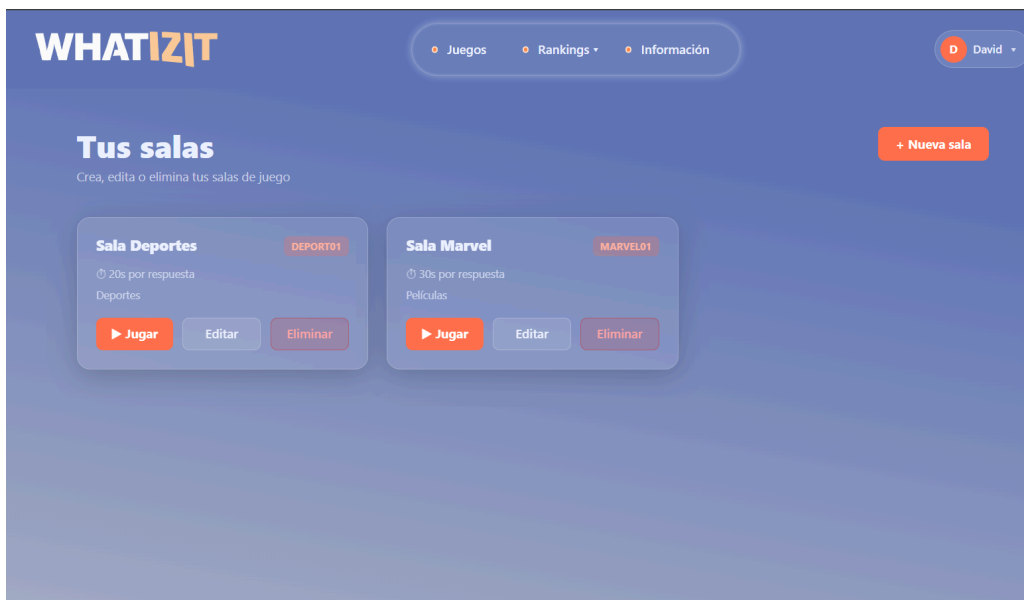
[Funciones](#) [Rankings](#) [FAQ](#) [Contacto](#)



Página de juego:



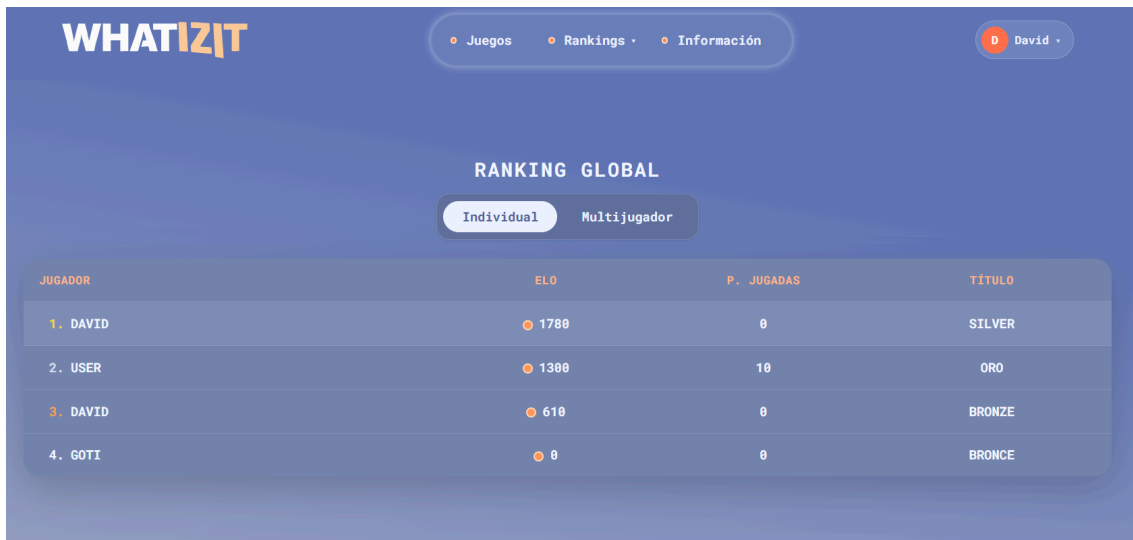
Esta es la página de creación de salas donde te salen las salas creadas y para agregar salas





Página de ranking:

Esta es la página de ranking global donde salen un top 50 mejores jugadores.



JUGADOR	ELO	P. JUGADAS	TÍTULO
1. DAVID	1780	0	SILVER
2. USER	1300	10	ORO
3. DAVID	610	0	BRONZE
4. GOTI	0	0	BRONCE

Y esta es la página de ranking global por categoría donde puedes filtrar por categoría y por top



#	Usuario	Puntuación	Partidas	Media
1	USER	1300	10	130.0

Página de información:

Y esta la pagina de información donde explicamos el funcionamiento del juego del ranking y las normas del juego para que haya un juego limpio y fluido.



[Juegos](#) [Rankings](#) [Información](#)

David

INFORMACIÓN DEL JUEGO

Consulta cómo jugar, normas y ranking.

[Cómo jugar](#) [Normas](#) [Ranking](#)

Información

¿Cómo se juega?

Sigue estos pasos y empieza a competir en tiempo real.

Pasos del juego

- 1 Entra en una sala de cualquier categoría.
- 2 Se mostrará una imagen en pantalla.
- 3 Escribe lo que crees que representa lo más rápido posible.
- 4 Cuanto antes aciertes, más puntos obtendrás.
- 5 Compite contra otros jugadores en tiempo real.
- 6 Sube posiciones en el ranking global y por categorías.

Claves

- ⚡ Velocidad + precisión = más puntos.
- 👤 Compites contra otros jugadores en tiempo real.
- 📊 La constancia te hace subir posiciones.

Ir a jugar



[Juegos](#) [Rankings](#) [Información](#)

David

INFORMACIÓN DEL JUEGO

Consulta cómo jugar, normas y ranking.

[Cómo jugar](#) [Normas](#) [Ranking](#)

Información

Normas del juego

Juego limpio para garantizar una experiencia justa para todos.

Reglas principales

- ✓ No hacer trampas ni usar herramientas externas.
- ✓ Respetar a los demás jugadores.
- ✓ No enviar respuestas ofensivas o inapropiadas.
- ✓ Jugar de forma limpia y competitiva.
- ✓ El incumplimiento puede suponer penalizaciones o bloqueo.

Aviso

- ⚠ El incumplimiento puede suponer penalizaciones o bloqueo.
- 👤 Respetar a los demás jugadores.
- 🚫 No se permiten respuestas ofensivas.

Ver salas disponibles

WHATIZIT
Juegos
Rankings
Información
David

INFORMACIÓN DEL JUEGO

Consulta cómo jugar, normas y ranking.

Cómo jugar
Normas
Ranking

¿Cómo funciona el ranking?

El ranking premia rapidez, precisión y constancia en cada partida.

Puntos por partida

Cada partida suma puntos según tu resultado.

Rapidez y precisión

Responder antes y acertar aumenta la puntuación.

Constancia

Jugar más te da más opciones de subir al Top.

Funcionamiento

- Los jugadores obtienen puntos en cada partida.
- La puntuación depende de la rapidez y precisión.
- Cuanto más partidas juegues, más posibilidades de subir posiciones.
- Los mejores jugadores aparecerán en el Top del ranking.

Existen rankings:

Global
Por categoría

Cómo subir posiciones

- Juega de forma regular.
- Prioriza respuestas rápidas pero correctas.
- Participa en varias categorías para mejorar tu nivel global.

Ver ranking

Comprobaciones thunder client rutas API:

Rutas públicas (sin auth)

GET /public/categories

THUNDER CLIENT
127.0.0.1:8000/api/catego...

New Request
Activity
Collections
Env
filter activity

GET 127.0.0.1:8000/api/categories just now
GET 127.0.0.1:8000/api/categories ... 4 mins ago
GET 127.0.0.1:8000/users just now
GET 127.0.0.1:8000/api/categories/id just now
GET 127.0.0.1:8000/api/salas 3 months ago
DEL 127.0.0.1:8000/api/salas 3 months ago
POST 127.0.0.1:8000/api/salas.admi... 3 months ago
PUT 127.0.0.1:8000/api/salas 3 months ago
GET 127.0.0.1:8000/api/salas/id 3 months ago
POST 127.0.0.1:8000/api/salas.user 3 months ago
POST 127.0.0.1:8000/login 3 months ago

GET http://127.0.0.1:8000/api/public/categories Send
Query Headers Auth Body Tests Pre Run

HTTP Headers
User-Agent Thunder Client (https://www.thunderclient.com)
Accept application/json
Authorization Bearer 106jcl799HbbDWacR4NAH9RLx51D5Gqkcpnrk4r
header value

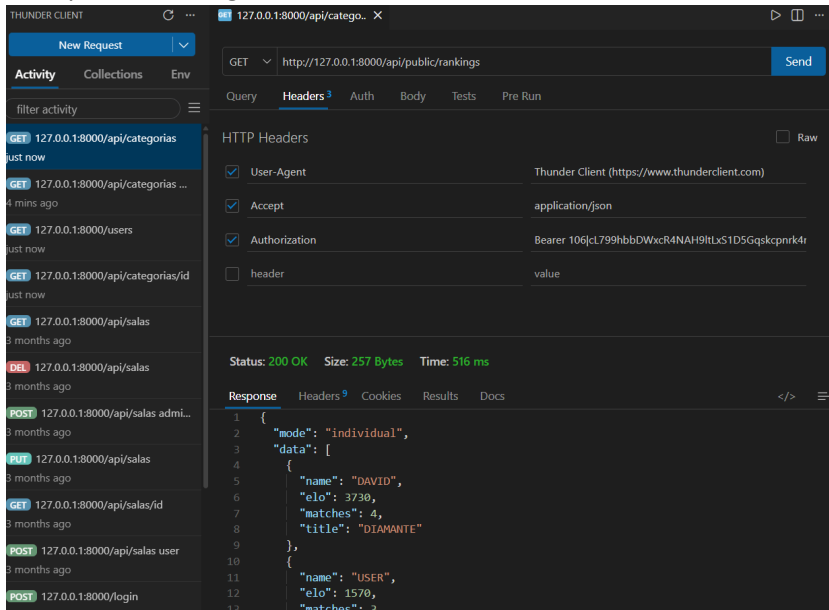
Status: 200 OK Size: 946 Bytes Time: 294 ms
Response Headers Cookies Results Docs

```

1 {
2   "data": [
3     {
4       "id": 2,
5       "slug": "deportes",
6       "name": "Deportes",
7       "description": "¿Conoces a los mejores atletas del mundo? Adivina deportistas, equipos y momentos legendarios del deporte internacional.",
8       "image": "/images/deportes.webp"
9     },
10    {
11      "id": 3,
12      "slug": "realizaci..."
13    }
14  ]
15 }

```

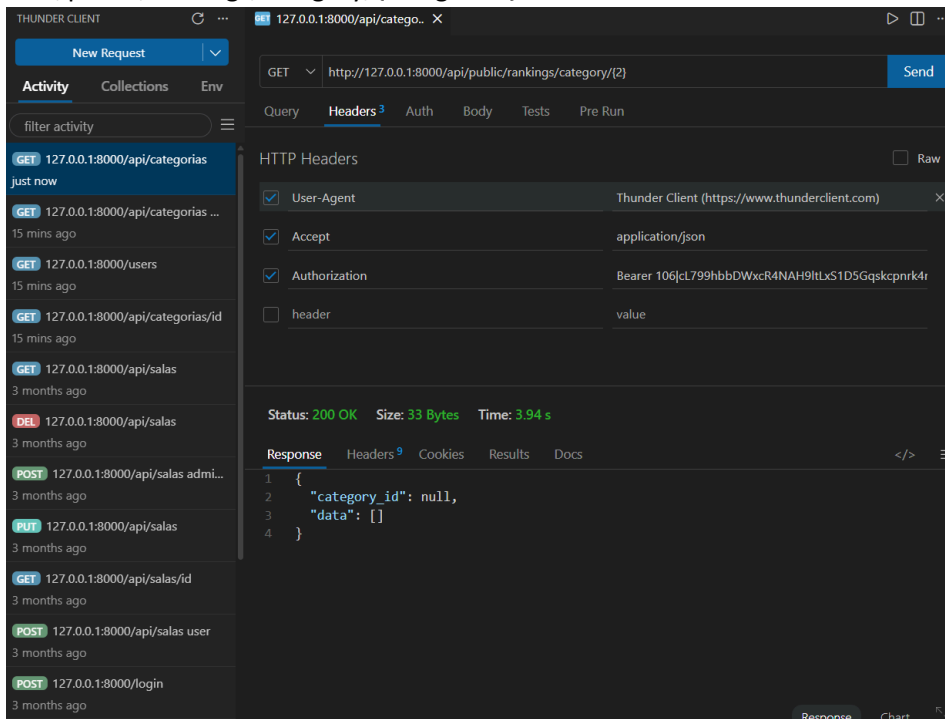
GET /public/rankings



Thunder Client interface showing a GET request to `http://127.0.0.1:8000/api/public/rankings`. The response is a JSON object with status 200 OK, size 257 Bytes, and time 516 ms. The response body shows a list of rankings for 'DAVID' and 'USER'.

```
1 {
2   "mode": "individual",
3   "data": [
4     {
5       "name": "DAVID",
6       "elo": 3730,
7       "matches": 4,
8       "title": "DIAMANTE"
9     },
10    {
11      "name": "USER",
12      "elo": 1570,
13      "matches": 3
14    }
15  ]
16 }
```

GET /public/rankings/category/{categoria?}



Thunder Client interface showing a GET request to `http://127.0.0.1:8000/api/public/rankings/category/{2}`. The response is a JSON object with status 200 OK, size 33 Bytes, and time 3.94 s. The response body shows a list of rankings for 'category_id': null.

```
1 {
2   "category_id": null,
3   "data": [
4     ]
5 }
```

Rutas protegidas (con auth:sanctum)

Admin/Usuarios



POST /admin/player-stats/reset

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST 127.0.0.1:8000/api/categorias just now

GET 127.0.0.1:8000/api/categorias ... 15 mins ago

GET 127.0.0.1:8000/users 15 mins ago

GET 127.0.0.1:8000/api/categorias/id 15 mins ago

GET 127.0.0.1:8000/api/salas 3 months ago

DEL 127.0.0.1:8000/api/salas 3 months ago

POST 127.0.0.1:8000/api/salas admin... 3 months ago

PUT 127.0.0.1:8000/api/salas 3 months ago

GET 127.0.0.1:8000/api/salas/id 3 months ago

POST 127.0.0.1:8000/api/salas user 3 months ago

POST 127.0.0.1:8000/login 3 months ago

POST http://127.0.0.1:8000/api/admin/player-stats/reset Send

Query Headers 3 Auth Body Tests Pre Run

HTTP Headers

☒ User-Agent Thunder Client (https://www.thunderclient.com)

☒ Accept application/json

☒ Authorization Bearer 106jcl799hbbDWxcR4NAH9ltXs1D5Ggskcpnrk4r

☐ header value

Status: 200 OK Size: 91 Bytes Time: 5.66 s

Response Headers 9 Cookies Results Docs

```
1 {
2   "message": "Las estadísticas de todos los jugadores se reiniciaron correctamente."
3 }
```

GET|POST /users

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET 127.0.0.1:8000/api/categorias just now

GET 127.0.0.1:8000/api/categorias ... 15 mins ago

GET 127.0.0.1:8000/users 15 mins ago

GET 127.0.0.1:8000/api/categorias/id 15 mins ago

GET 127.0.0.1:8000/api/salas 3 months ago

DEL 127.0.0.1:8000/api/salas 3 months ago

POST 127.0.0.1:8000/api/salas admin... 3 months ago

PUT 127.0.0.1:8000/api/salas 3 months ago

GET 127.0.0.1:8000/api/salas/id 3 months ago

POST 127.0.0.1:8000/api/salas user 3 months ago

POST 127.0.0.1:8000/login 3 months ago

GET http://127.0.0.1:8000/api/users Send

Query Headers 3 Auth Body Tests Pre Run

HTTP Headers

☒ User-Agent Thunder Client (https://www.thunderclient.com)

☒ Accept application/json

☒ Authorization Bearer 106jcl799hbbDWxcR4NAH9ltXs1D5Ggskcpnrk4r

☐ header value

Status: 200 OK Size: 1.47 KB Time: 1.48 s

Response Headers 9 Cookies Results Docs

```
1 {
2   "data": [
3     {
4       "id": 6,
5       "alias": null,
6       "name": "LA VACA",
7       "surname": "LOLA",
8       "surname2": "LECHERA",
9       "email": "joaimesf@bf.cat",
10      "elo": 0,
11      "partidas_jugadas": 0,
12      "titulo": "Bronce",
13      "rolas": {}
14     }
15   ]
16 }
```

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST 127.0.0.1:8000/api/categorias just now

GET 127.0.0.1:8000/api/categorias ... 15 mins ago

GET 127.0.0.1:8000/users 15 mins ago

GET 127.0.0.1:8000/api/categorias/id 15 mins ago

GET 127.0.0.1:8000/api/salas 3 months ago

DEL 127.0.0.1:8000/api/salas 3 months ago

POST 127.0.0.1:8000/api/salas admin... 3 months ago

PUT 127.0.0.1:8000/api/salas 3 months ago

GET 127.0.0.1:8000/api/salas/id 3 months ago

POST 127.0.0.1:8000/api/salas user 3 months ago

POST 127.0.0.1:8000/login 3 months ago

POST http://127.0.0.1:8000/api/users Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "Admin",
3   "surname": "David",
4   "email": "admin@demo.com",
5   "password": "12345678"
6 }
```

Status: 201 Created Size: 203 Bytes Time: 1.22 s

Response Headers 9 Cookies Results Docs

```
1 {
2   "data": {
3     "id": 9,
4     "alias": null,
5     "name": "Admin",
6     "surname": "David",
7     "surname2": null,
8     "email": "admin@demo.com",
9     "elo": 0,
10    "partidas_jugadas": 0,
11    "titulo": null,
12    "roles": [],
13    "avatar": null
14  }
15 }
```



GET|PUT|DELETE /users/{user}

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET 127.0.0.1:8000/api/categorias just now

GET 127.0.0.1:8000/api/categorias ... 15 mins ago

GET 127.0.0.1:8000/users 15 mins ago

GET 127.0.0.1:8000/api/categorias/id 15 mins ago

GET 127.0.0.1:8000/api/salas 3 months ago

DELETE 127.0.0.1:8000/api/salas 3 months ago

POST 127.0.0.1:8000/api/salas admin... 3 months ago

PUT 127.0.0.1:8000/api/salas 3 months ago

GET 127.0.0.1:8000/api/salas/id 3 months ago

POST 127.0.0.1:8000/api/salas user 3 months ago

POST 127.0.0.1:8000/login 3 months ago

127.0.0.1:8000/api/catego.. X

GET http://127.0.0.1:8000/api/users/8 Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "name": "Admin",
3   "surname1": "Apellidoi",
4   "email": "admin@demo.com",
5   "password": "12345678"
6 }
```

Status: 200 OK Size: 203 Bytes Time: 21.16 s

Response Headers Cookies Results Docs

```
1 {
2   "data": {
3     "id": 8,
4     "alias": null,
5     "name": "Admin",
6     "surname1": "David",
7     "surname2": null,
8     "email": "admin@demo.com",
9     "elo": 0,
10    "partidas_jugadas": 0,
11    "titulo": null,
12    "roles": [],
13    "avatar": null
14  }
15 }
```

Response Chart

127.0.0.1:8000/api/catego.. X

PUT http://127.0.0.1:8000/api/users/8 Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "name": "Admin",
3   "email": "admin@example.com",
4   "surname1": "Apellidoi",
5   "surname2": "Apellido2",
6   "password": "secret123",
7   "role_id": 1
8 }
```

Status: 200 OK Size: 276 Bytes Time: 2.95 s

Response Headers Cookies Results Docs

```
1 {
2   "data": {
3     "id": 8,
4     "alias": null,
5     "name": "Admin",
6     "surname1": "Apellidoi",
7     "surname2": "Apellido2",
8     "email": "admin@example.com",
9     "elo": 0,
10    "partidas_jugadas": 0,
11    "titulo": null,
12    "roles": [
13      f
14    ]
15  }
16 }
```

Response Chart

127.0.0.1:8000/users X 127.0.0.1:8000/api/catego..

DELETE http://127.0.0.1:8000/api/users/9 Send

Query Headers Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒ User-Agent Thunder Client (https://www.thunderclient.com)

☒ Accept application/json

☒ Authorization Bearer 106jcl799hbbDWxcR4NAH9ltLxS1D5Gqskcpnrk4r

☐ header value

Status: 204 No Content Size: 0 Bytes Time: 754 ms

Response Headers Cookies Results Docs

1

GET | POST /categorias

GET ▼ http://127.0.0.1:8000/api/categorias Send

Query Headers ³ Auth Body Tests Pre Run

HTTP Headers Raw

☒	User-Agent	Thunder Client (https://www.thunderclient.com)
☒	Accept	application/json
☒	Authorization	Bearer 106jcl799hbbDWxcR4NAH9ltLxS1D5Gqskcpnrk4r
☐	header	value

Status: **200 OK** Size: **3.17 KB** Time: **928 ms**

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id": 2,
6       "nombre": "Deportes",
7       "imagen": "/images/deportes.webp",
8       "descripcion": "¿Conoces a los mejores atletas del mundo? Adivina deportistas,
9         equipos y momentos legendarios del deporte internacional.",
10      "salas": [
11        {
12          "id": 2,
13          "nombre": "Sala Deportes"
```

127.0.0.1:8000/users X 127.0.0.1:8000/api/catego.. ▶ □ ⋮

POST ▼ http://127.0.0.1:8000/api/categorias Send

Query Headers ³ Auth **Body** ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "nombre": "Naturaleza",
3   "descripcion": "Descripción opcional",
4   "imagen": "/images/cat.png",
5   "slug": "naturaleza"
6 }
```

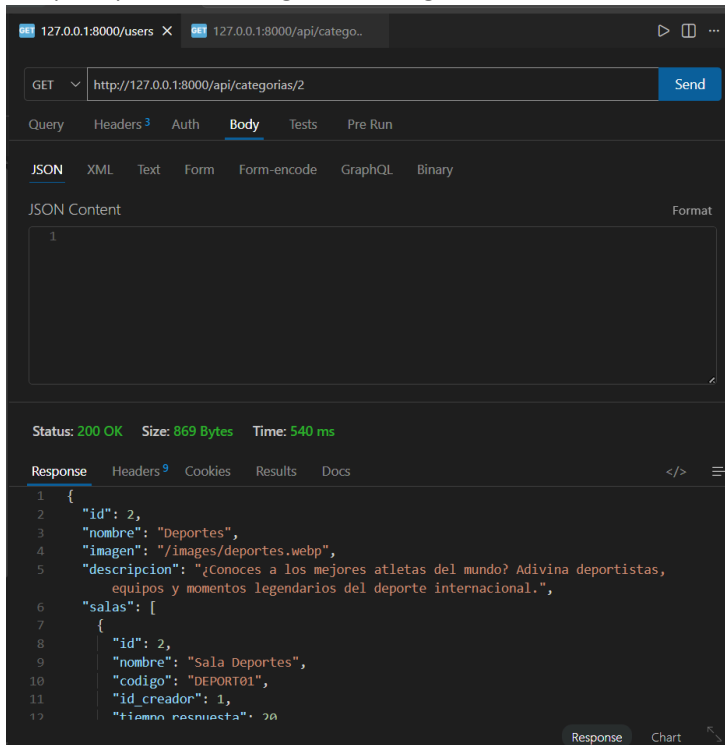
Status: **201 Created** Size: **33 Bytes** Time: **831 ms**

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "nombre": "Naturaleza",
3   "id": 9
4 }
```

Response Chart ↗

GET|PUT|DELETE /categorias/{categoria}



127.0.0.1:8000/users x 127.0.0.1:8000/api/catego..

GET http://127.0.0.1:8000/api/categorias/2 Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

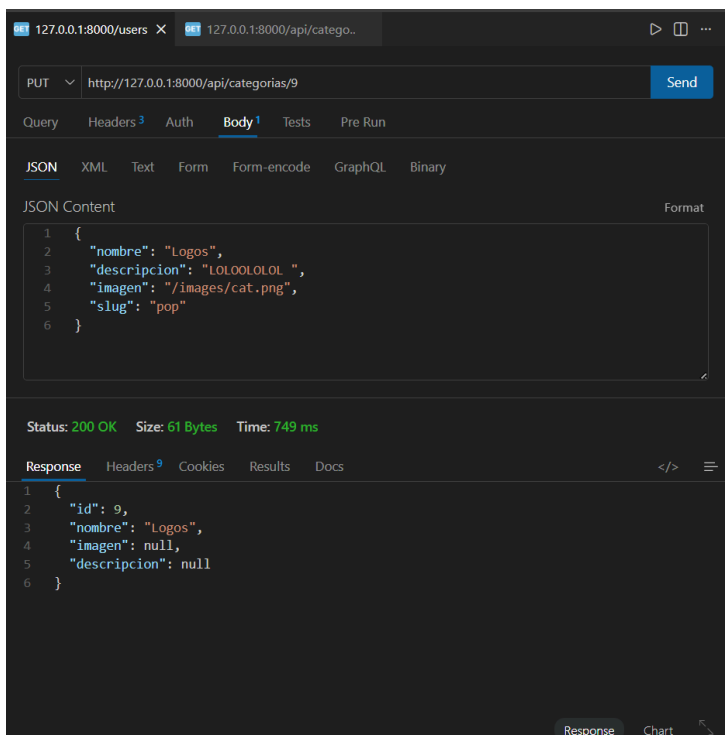
```
1
```

Status: 200 OK Size: 869 Bytes Time: 540 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 2,
3   "nombre": "Deportes",
4   "imagen": "/imagenes/deportes.webp",
5   "descripcion": "¿Conoces a los mejores atletas del mundo? Adivina deportistas,
6     equipos y momentos legendarios del deporte internacional.",
7   "salas": [
8     {
9       "id": 2,
10      "nombre": "Sala Deportes",
11      "codigo": "DEPORT01",
12      "id_creador": 1,
13      "tiempo_respuesta": 20
```

Response Chart



127.0.0.1:8000/users x 127.0.0.1:8000/api/catego..

PUT http://127.0.0.1:8000/api/categorias/9 Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "nombre": "Logos",
3   "descripcion": "LOLOOLOLOL ",
4   "imagen": "/imagenes/cat.png",
5   "slug": "pop"
6 }
```

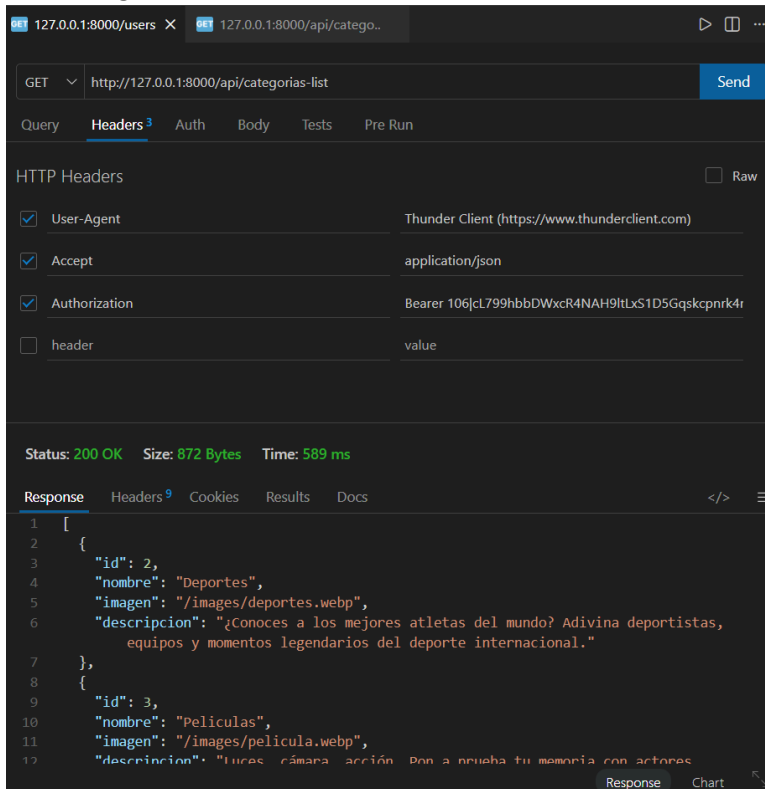
Status: 200 OK Size: 61 Bytes Time: 749 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 9,
3   "nombre": "Logos",
4   "imagen": null,
5   "descripcion": null
6 }
```

Response Chart

GET /categorias-list



Thunder Client interface showing a GET request to `http://127.0.0.1:8000/api/categorias-list`. The request is successful with a 200 OK status. The response is a JSON array with two category objects.

Query: GET `http://127.0.0.1:8000/api/categorias-list` [Send](#)

Headers: 3

Header	Value
User-Agent	Thunder Client (https://www.thunderclient.com)
Accept	application/json
Authorization	Bearer 106jcl799hbbDWxcR4NAH9ltLxS1D5Gskcpnrk4r
header	value

Status: 200 OK **Size:** 872 Bytes **Time:** 589 ms

Response: 9

```
1 [
2   {
3     "id": 2,
4     "nombre": "Deportes",
5     "imagen": "/images/deportes.webp",
6     "descripcion": "¿Conoces a los mejores atletas del mundo? Adivina deportistas,
7     equipos y momentos legendarios del deporte internacional."
8   },
9   {
10    "id": 3,
11    "nombre": "Películas",
12    "imagen": "/images/pelicula.webp",
13    "descripcion": "Lucas cámara acción. Pon a prueba tu memoria con actores
```

GET|POST /salas

127.0.0.1:8000/users x 127.0.0.1:8000/api/catego..

GET http://127.0.0.1:8000/api/salas Send

Query Headers 3 Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒ User-Agent	Thunder Client (https://www.thunderclient.com)
☒ Accept	application/json
☒ Authorization	Bearer 106jcl799hbbDWxcR4NAH9tLxS1D5Gqskcprnk4r
☐ header	value

Status: 200 OK Size: 4.08 KB Time: 912 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id": 15,
6       "nombre": "holai",
7       "codigo": "48284",
8       "id_creador": 1,
9       "tiempo_respuesta": 30,
10      "fecha_creacion": "2026-05-13 17:32:13",
11      "categorias": [
12        {
13          "id": 2
```

127.0.0.1:8000/users x 127.0.0.1:8000/api/catego..

POST http://127.0.0.1:8000/api/salas Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

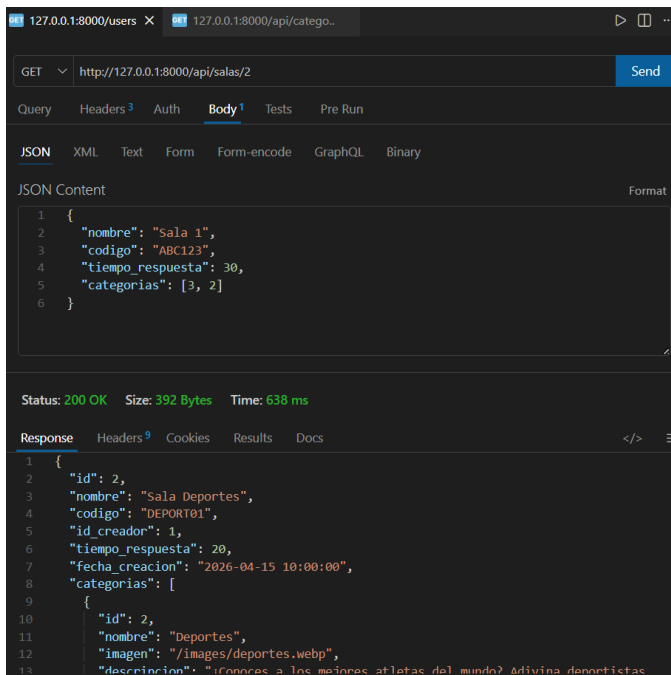
```
1 {
2   "nombre": "Sala 1",
3   "codigo": "ABC123",
4   "tiempo_respuesta": 30,
5   "categorias": [3, 2]
6 }
```

Status: 201 Created Size: 613 Bytes Time: 765 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "nombre": "Sala 1",
3   "codigo": "ABC123",
4   "tiempo_respuesta": 30,
5   "id_creador": 1,
6   "id": 16,
7   "categorias": [
8     {
9       "id": 2,
10      "nombre": "Deportes",
11      "imagen": "/images/deportes.webp",
12      "descripcion": "¿Conoces a los mejores atletas del mundo? Adivina deportistas,
equinos y momentos legendarios del deporte internacional "
```

GET|PUT|DELETE /salas/{sala}



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

GET http://127.0.0.1:8000/api/salas/2 Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

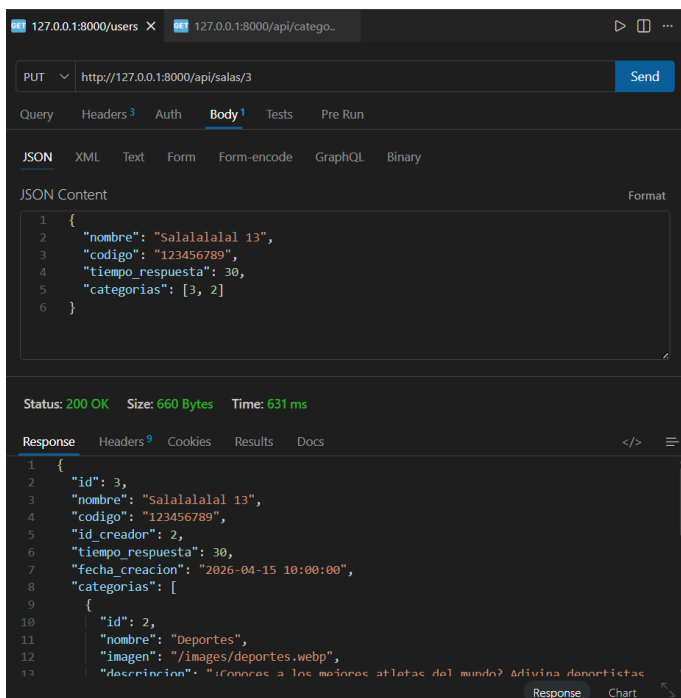
JSON Content Format

```
1 {
2   "nombre": "Sala 1",
3   "codigo": "ABC123",
4   "tiempo_respuesta": 30,
5   "categorias": [3, 2]
6 }
```

Status: 200 OK Size: 392 Bytes Time: 638 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 2,
3   "nombre": "Sala Deportes",
4   "codigo": "DEPORT01",
5   "id_creador": 1,
6   "tiempo_respuesta": 20,
7   "fecha_creacion": "2026-04-15 10:00:00",
8   "categorias": [
9     {
10      "id": 2,
11      "nombre": "Deportes",
12      "imagen": "/images/deportes.webp",
13      "descripcion": "¿Conoces a las mejores atletas del mundo? Adivina deportivas"
```



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

PUT http://127.0.0.1:8000/api/salas/3 Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

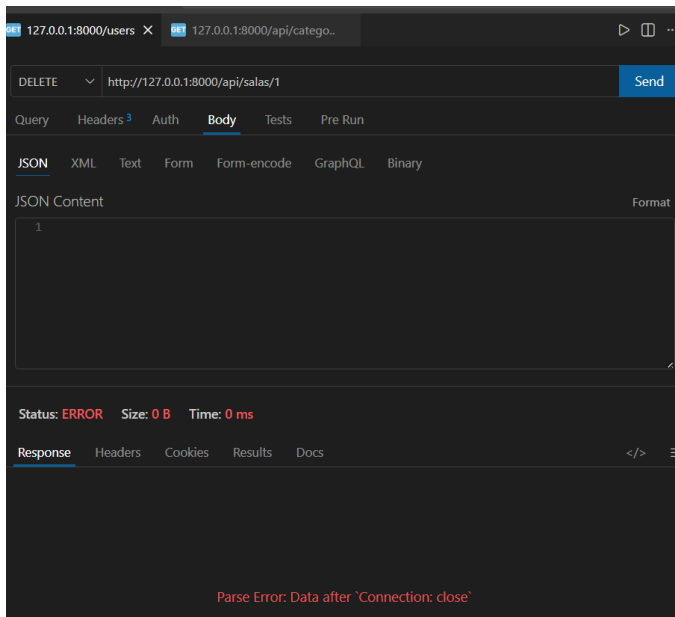
JSON Content Format

```
1 {
2   "nombre": "Salalalalal 13",
3   "codigo": "123456789",
4   "tiempo_respuesta": 30,
5   "categorias": [3, 2]
6 }
```

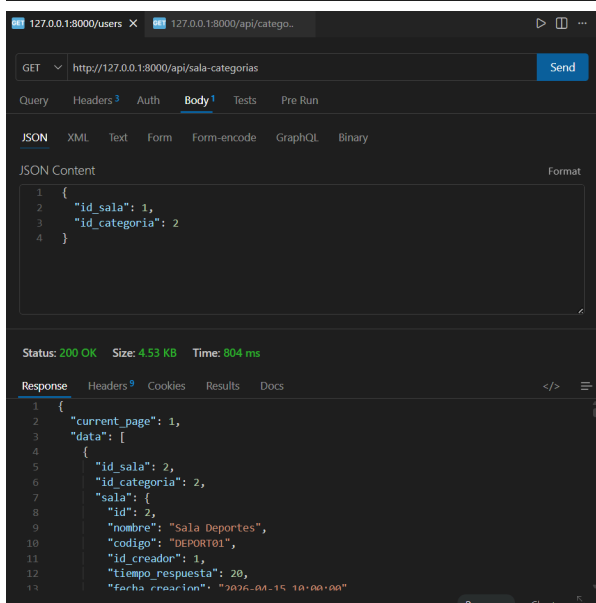
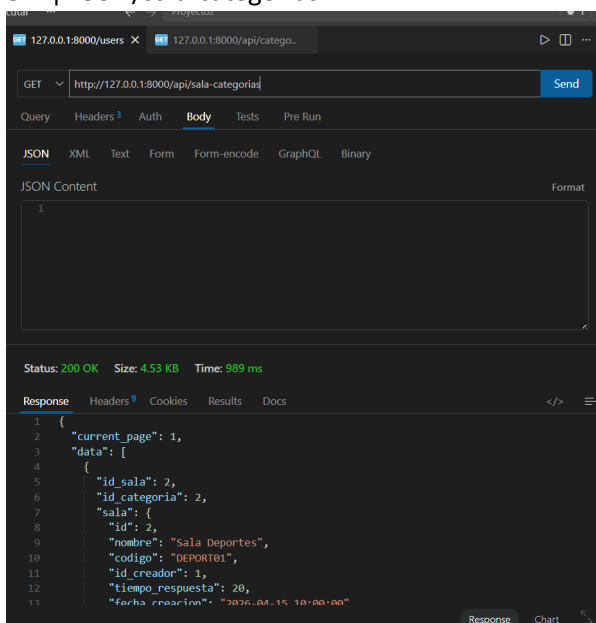
Status: 200 OK Size: 660 Bytes Time: 631 ms

Response Headers 9 Cookies Results Docs

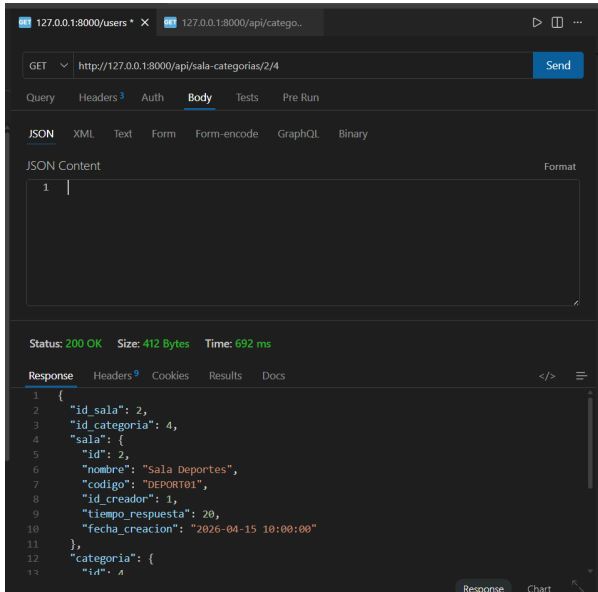
```
1 {
2   "id": 3,
3   "nombre": "Salalalalal 13",
4   "codigo": "123456789",
5   "id_creador": 2,
6   "tiempo_respuesta": 30,
7   "fecha_creacion": "2026-04-15 10:00:00",
8   "categorias": [
9     {
10      "id": 2,
11      "nombre": "Deportes",
12      "imagen": "/images/deportes.webp",
13      "descripcion": "¿Conoces a las mejores atletas del mundo? Adivina deportivas"
```



GET|POST /sala-categorias



GET|PUT|DELETE /sala-categorias/{id_sala}/{id_categoria}



127.0.0.1:8000/users x 127.0.0.1:8000/api/catego...

GET http://127.0.0.1:8000/api/sala-categorias/2/4 Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

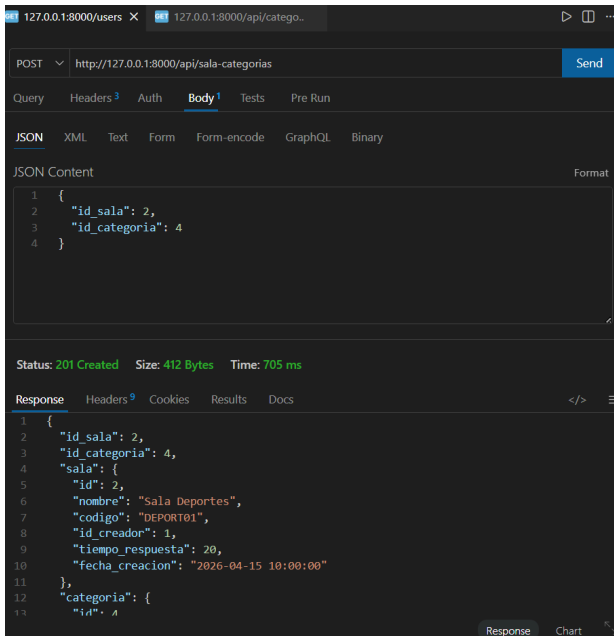
```
1 {
```

Status: 200 OK Size: 412 Bytes Time: 692 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "id_sala": 2,
3   "id_categoria": 4,
4   "sala": {
5     "id": 2,
6     "nombre": "Sala Deportes",
7     "codigo": "DEPORT01",
8     "id_creador": 1,
9     "tiempo_respuesta": 20,
10    "fecha_creacion": "2026-04-15 10:00:00"
11  },
12  "categoria": {
13    "id": 4
```

Response Chart ↗



127.0.0.1:8000/users x 127.0.0.1:8000/api/catego...

POST http://127.0.0.1:8000/api/sala-categorias Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id_sala": 2,
3   "id_categoria": 4
4 }
```

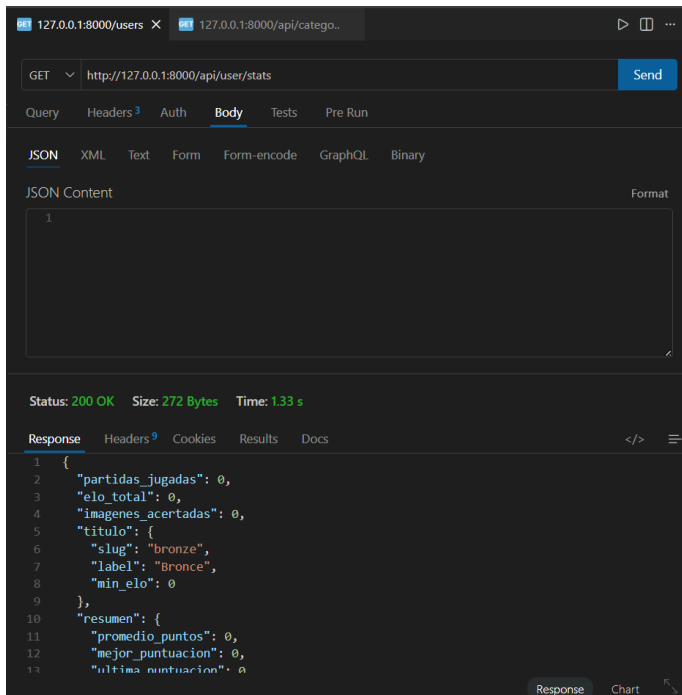
Status: 201 Created Size: 412 Bytes Time: 705 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "id_sala": 2,
3   "id_categoria": 4,
4   "sala": {
5     "id": 2,
6     "nombre": "Sala Deportes",
7     "codigo": "DEPORT01",
8     "id_creador": 1,
9     "tiempo_respuesta": 20,
10    "fecha_creacion": "2026-04-15 10:00:00"
11  },
12  "categoria": {
13    "id": 4
```

Response Chart ↗

GET /user/stats



GET `http://127.0.0.1:8000/api/user/stats` Send

Query Headers ³ Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

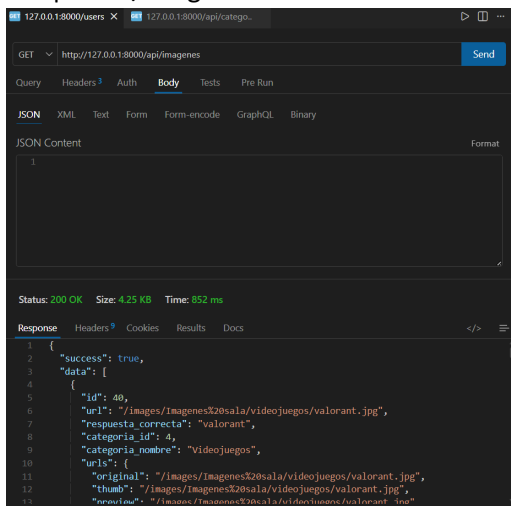
Status: 200 OK Size: 272 Bytes Time: 1.33 s

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "partidas_jugadas": 0,
3   "elo_total": 0,
4   "imagenes_acertadas": 0,
5   "titulo": {
6     "slug": "bronze",
7     "label": "Bronce",
8     "min_elo": 0
9   },
10  "resumen": {
11    "promedio_puntos": 0,
12    "mejor_puntuacion": 0,
13    "ultima_puntuacion": 0
14  }
```

Response Chart

GET|POST /imagenes



GET `http://127.0.0.1:8000/api/imagenes` Send

Query Headers ³ Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

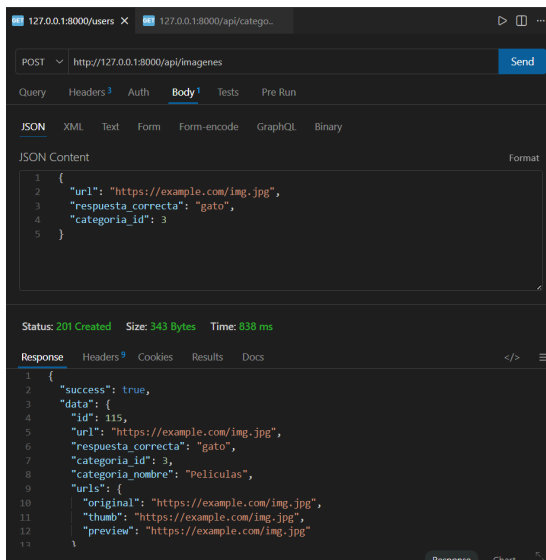
JSON Content Format

```
1
```

Status: 200 OK Size: 4.25 KB Time: 852 ms

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 40,
6       "url": "/images/imagenes320sala/videojuegos/valorant.jpg",
7       "respuesta_correcta": "valorant",
8       "categoria_id": 4,
9       "categoria_nombre": "Videojuegos",
10      "urls": {
11        "original": "/images/imagenes320sala/videojuegos/valorant.jpg",
12        "thumb": "/images/imagenes320sala/videojuegos/valorant.jpg",
13        "preview": "/images/imagenes320sala/videojuegos/valorant.jpg"
14      }
15     }
16   ]
17 }
```



POST `http://127.0.0.1:8000/api/imagenes` Send

Query Headers ³ Auth **Body** ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "url": "https://example.com/img.jpg",
3   "respuesta_correcta": "gato",
4   "categoria_id": 3
5 }
```

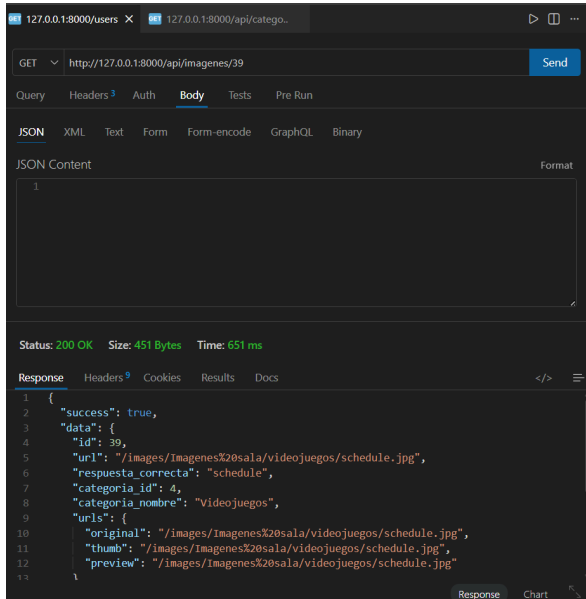
Status: 201 Created Size: 343 Bytes Time: 838 ms

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "success": true,
3   "data": {
4     "id": 115,
5     "url": "https://example.com/img.jpg",
6     "respuesta_correcta": "gato",
7     "categoria_id": 3,
8     "categoria_nombre": "Películas",
9     "urls": {
10      "original": "https://example.com/img.jpg",
11      "thumb": "https://example.com/img.jpg",
12      "preview": "https://example.com/img.jpg"
13    }
14   }
15 }
```

Response Chart

GET|PUT|DELETE /imagenes/{imagen}



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

GET http://127.0.0.1:8000/api/imagenes/39 Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

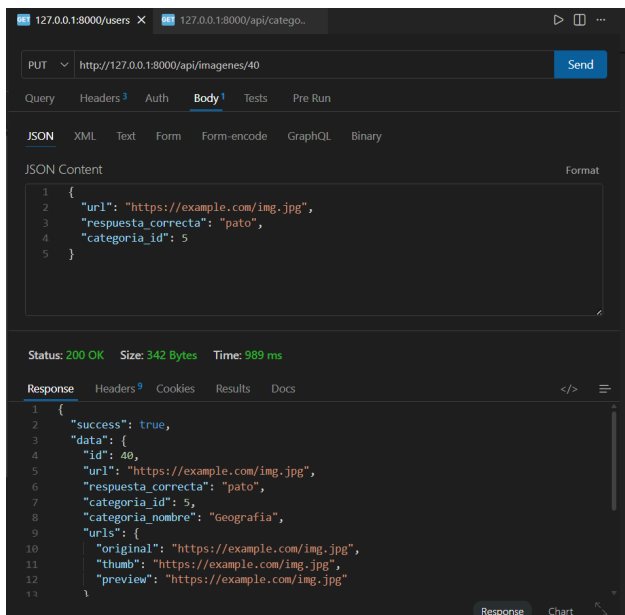
```
1
```

Status: 200 OK Size: 451 Bytes Time: 651 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "success": true,
3   "data": {
4     "id": 39,
5     "url": "/images/Imagenes%20sala/videojuegos/schedule.jpg",
6     "respuesta_correcta": "schedule",
7     "categoria_id": 4,
8     "categoria_nombre": "Videojuegos",
9     "urls": {
10      "original": "/images/Imagenes%20sala/videojuegos/schedule.jpg",
11      "thumb": "/images/Imagenes%20sala/videojuegos/schedule.jpg",
12      "preview": "/images/Imagenes%20sala/videojuegos/schedule.jpg"
13    }
14  }
15 }
```

Response Chart



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

PUT http://127.0.0.1:8000/api/imagenes/40 Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "url": "https://example.com/img.jpg",
3   "respuesta_correcta": "pato",
4   "categoria_id": 5
5 }
```

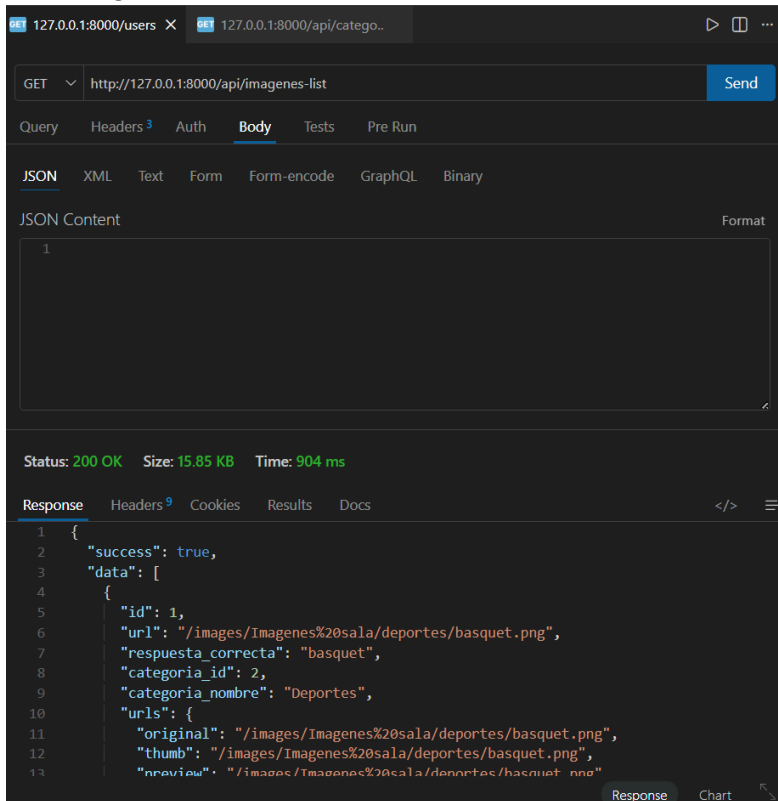
Status: 200 OK Size: 342 Bytes Time: 989 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "success": true,
3   "data": {
4     "id": 40,
5     "url": "https://example.com/img.jpg",
6     "respuesta_correcta": "pato",
7     "categoria_id": 5,
8     "categoria_nombre": "Geografia",
9     "urls": {
10      "original": "https://example.com/img.jpg",
11      "thumb": "https://example.com/img.jpg",
12      "preview": "https://example.com/img.jpg"
13    }
14  }
15 }
```

Response Chart

GET /imagenes-list

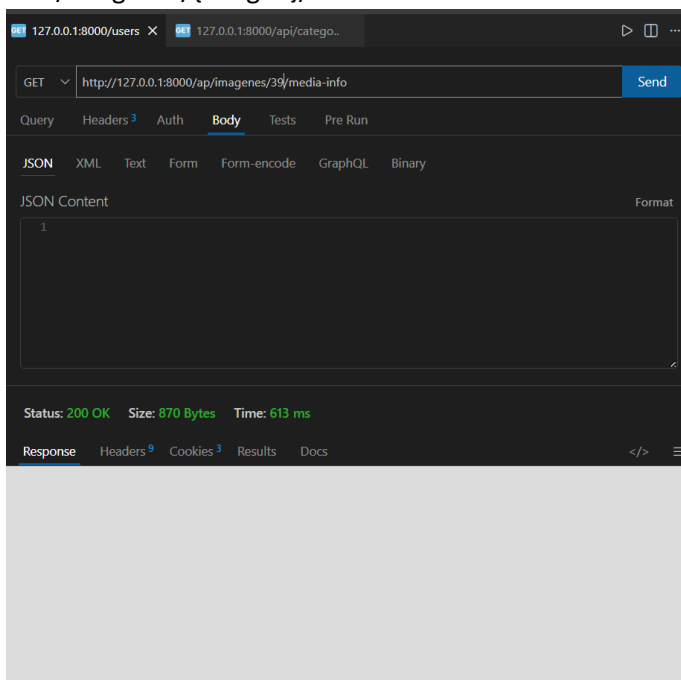


The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/imagenes-list
- Status:** 200 OK
- Size:** 15.85 KB
- Time:** 904 ms
- Response Body (JSON):**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "url": "/images/Imagenes%20sala/deportes/basquet.png",
      "respuesta_correcta": "basquet",
      "categoria_id": 2,
      "categoria_nombre": "Deportes",
      "urls": {
        "original": "/images/Imagenes%20sala/deportes/basquet.png",
        "thumb": "/images/Imagenes%20sala/deportes/basquet.png",
        "preview": "/images/Imagenes%20sala/deportes/basquet.png"
      }
    }
  ]
}
```

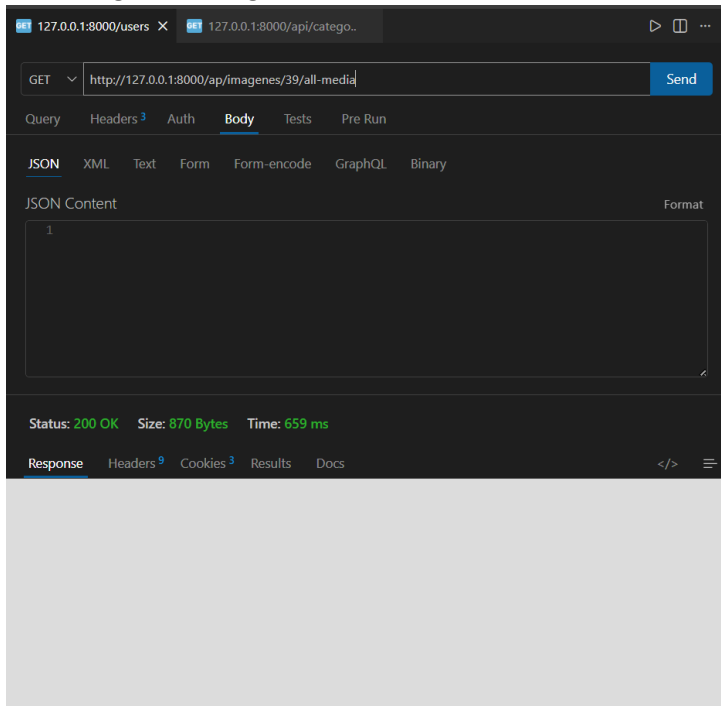
GET /imagenes/{imagen}/media-info



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/ap/imagenes/39/media-info
- Status:** 200 OK
- Size:** 870 Bytes
- Time:** 613 ms
- Response Body:** The response body is currently empty, indicated by a large grey rectangular area below the status bar.

GET /imagenes/{imagen}/all-media



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego..

GET http://127.0.0.1:8000/ap/imagenes/39/all-media Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

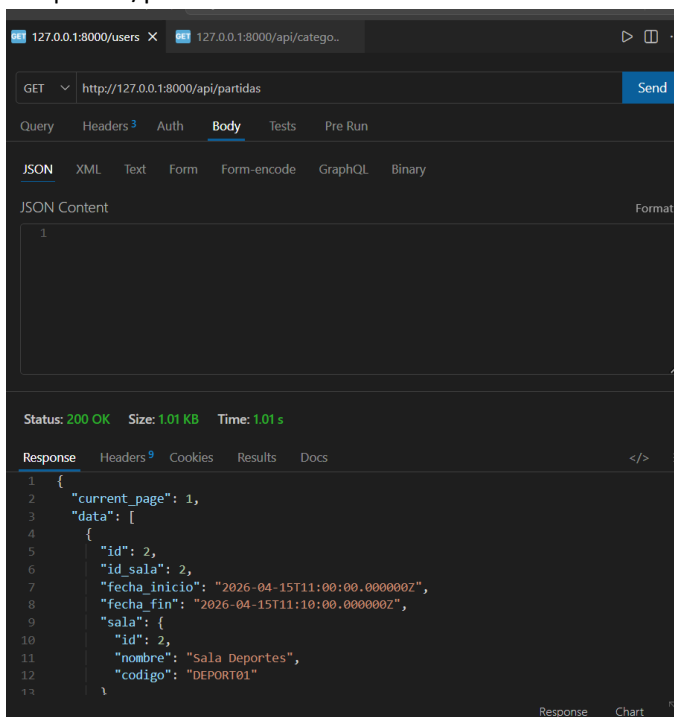
JSON Content Format

1

Status: 200 OK Size: 870 Bytes Time: 659 ms

Response Headers 9 Cookies 3 Results Docs

GET|POST /partidas



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego..

GET http://127.0.0.1:8000/api/partidas Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

1

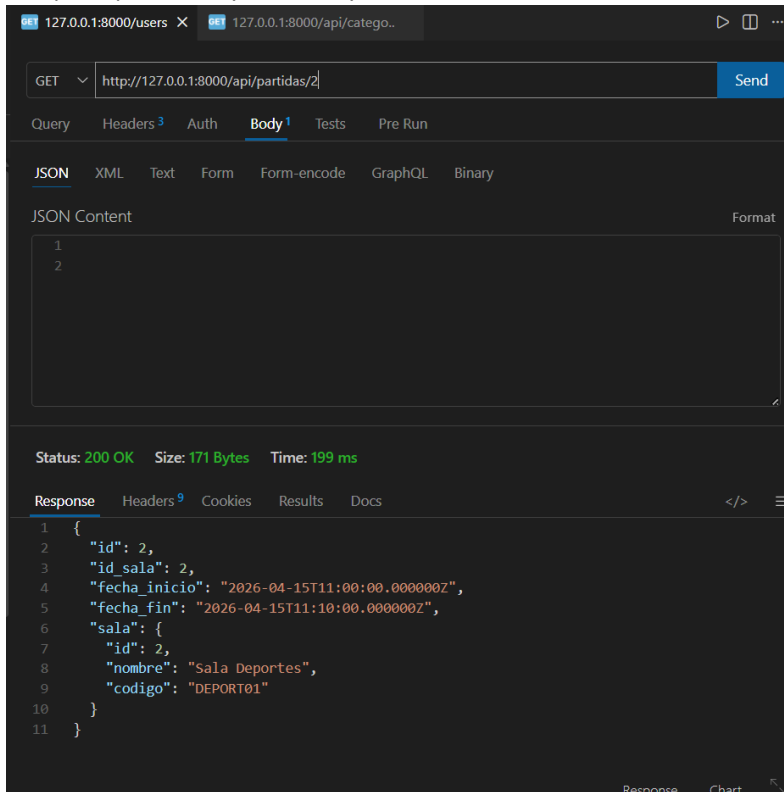
Status: 200 OK Size: 1.01 KB Time: 1.01 s

Response Headers 9 Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id": 2,
6       "id_sala": 2,
7       "fecha_inicio": "2026-04-15T11:00:00.000000Z",
8       "fecha_fin": "2026-04-15T11:10:00.000000Z",
9       "sala": {
10        "id": 2,
11        "nombre": "Sala Deportes",
12        "codigo": "DEPORT01"
13      }
14    }
15  ]
16 }
```

Response Chart

GET|PUT|DELETE /partidas/{partida}



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego..

GET Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

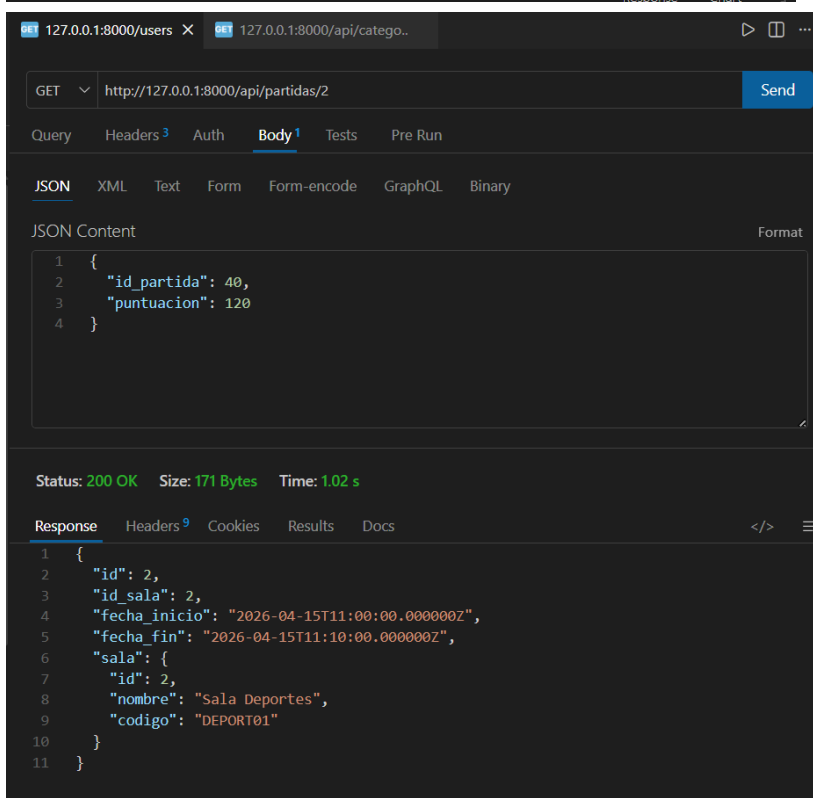
JSON Content Format

```
1
2
```

Status: 200 OK Size: 171 Bytes Time: 199 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 2,
3   "id_sala": 2,
4   "fecha_inicio": "2026-04-15T11:00:00.000000Z",
5   "fecha_fin": "2026-04-15T11:10:00.000000Z",
6   "sala": {
7     "id": 2,
8     "nombre": "Sala Deportes",
9     "codigo": "DEPORT01"
10  }
11 }
```



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego..

GET Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id_partida": 40,
3   "puntuacion": 120
4 }
```

Status: 200 OK Size: 171 Bytes Time: 1.02 s

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 2,
3   "id_sala": 2,
4   "fecha_inicio": "2026-04-15T11:00:00.000000Z",
5   "fecha_fin": "2026-04-15T11:10:00.000000Z",
6   "sala": {
7     "id": 2,
8     "nombre": "Sala Deportes",
9     "codigo": "DEPORT01"
10  }
11 }
```



GET /usuario-partidas

GET Send

Query Headers 3 Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

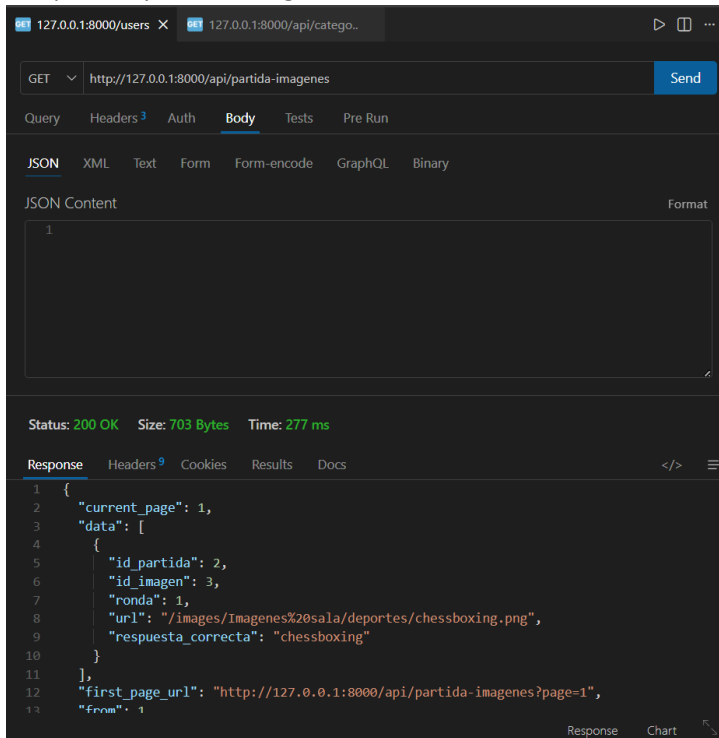
```
1
```

Status: 200 OK Size: 573 Bytes Time: 716 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [],
4   "first_page_url": "http://127.0.0.1:8000/api/usuario-partidas?page=1",
5   "from": null,
6   "last_page": 1,
7   "last_page_url": "http://127.0.0.1:8000/api/usuario-partidas?page=1",
8   "links": [
9     {
10      "url": null,
11      "label": "&laquo; Previous",
12      "page": null,
13      "active": false
```

GET|POST /partida-imagenes



GET `http://127.0.0.1:8000/api/partida-imagenes` Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

1

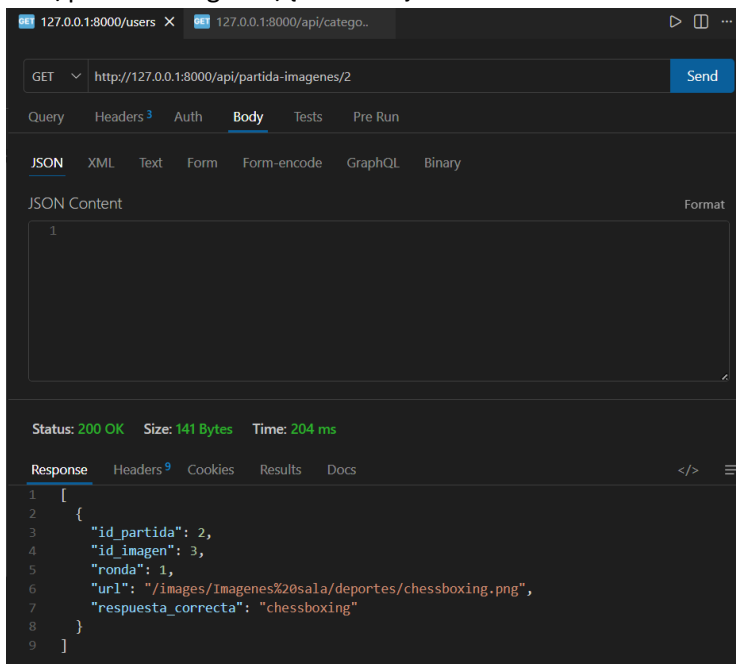
Status: 200 OK Size: 703 Bytes Time: 277 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id_partida": 2,
6       "id_imagen": 3,
7       "ronda": 1,
8       "url": "/images/Imagenes%20sala/deportes/chessboxing.png",
9       "respuesta_correcta": "chessboxing"
10    }
11  ],
12  "first_page_url": "http://127.0.0.1:8000/api/partida-imagenes?page=1",
13  "from": 1
14 }
```

Response Chart

GET /partida-imagenes/{idPartida}



GET `http://127.0.0.1:8000/api/partida-imagenes/2` Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

1

Status: 200 OK Size: 141 Bytes Time: 204 ms

Response Headers 9 Cookies Results Docs

```
1 [
2   {
3     "id_partida": 2,
4     "id_imagen": 3,
5     "ronda": 1,
6     "url": "/images/Imagenes%20sala/deportes/chessboxing.png",
7     "respuesta_correcta": "chessboxing"
8   }
9 ]
```

GET | POST /imagen-categorias

GET 127.0.0.1:8000/users X GET 127.0.0.1:8000/api/catego...

GET http://127.0.0.1:8000/api/imagen-categorias Send

Query Headers 3 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Status: 200 OK Size: 2.13 KB Time: 253 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id_imagen": 1,
6       "id_categoria": 2,
7       "url": "/images/Imagenes%20sala/deportes/basquet.png",
8       "categoria_nombre": "Deportes"
9     },
10    {
11      "id_imagen": 2,
12      "id_categoria": 2,
13      "url": "/images/Imagenes%20sala/deportes/huevo.png"
```

Response Chart

GET 127.0.0.1:8000/users X GET 127.0.0.1:8000/api/catego...

POST http://127.0.0.1:8000/api/imagen-categorias Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

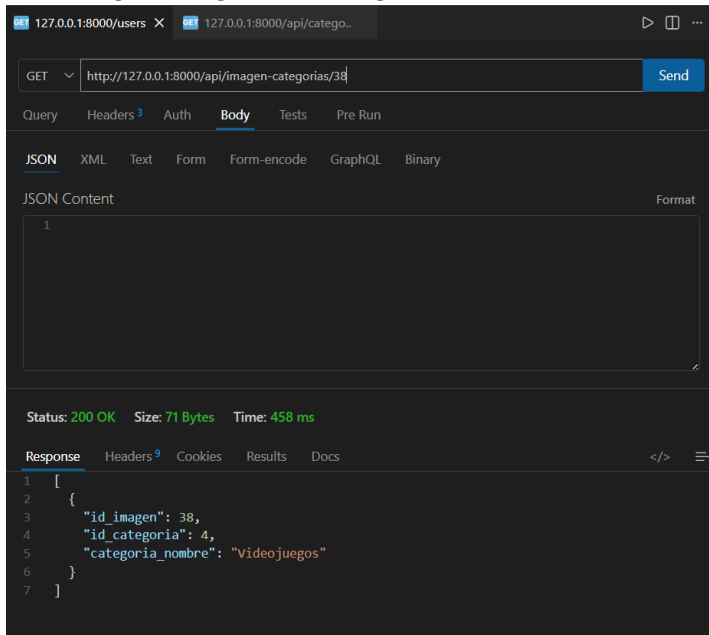
```
1 {
2   "id_imagen": 1,
3   "id_categoria": 5
4 }
```

Status: 201 Created Size: 35 Bytes Time: 197 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "id_imagen": 1,
3   "id_categoria": 5
4 }
```

GET /imagen-categorias/{idImagen}

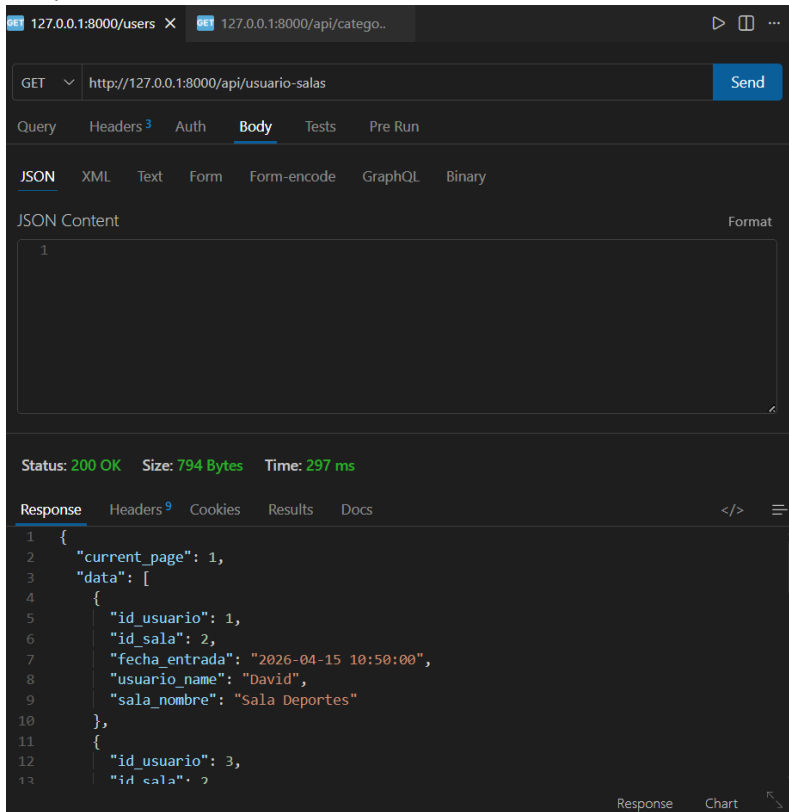


The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:8000/api/imagen-categorias/38`
- Method:** GET
- Body:** Empty (JSON Content)
- Status:** 200 OK
- Size:** 71 Bytes
- Time:** 458 ms
- Response:**

```
1 [
2   {
3     "id_imagen": 38,
4     "id_categoria": 4,
5     "categoria_nombre": "Videojuegos"
6   }
7 ]
```

GET|POST /usuario-salas



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

GET Send

Query Headers³ Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

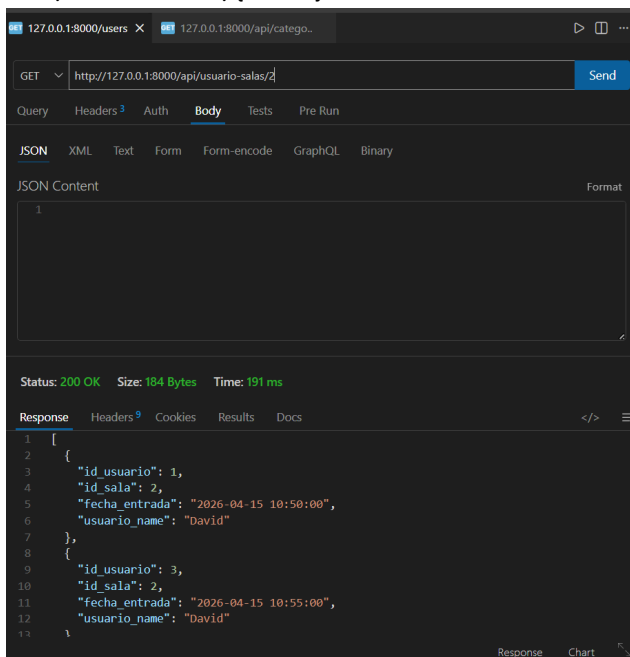
```
1
```

Status: 200 OK Size: 794 Bytes Time: 297 ms

Response Headers⁹ Cookies Results Docs

```
1 {
2   "current_page": 1,
3   "data": [
4     {
5       "id_usuario": 1,
6       "id_sala": 2,
7       "fecha_entrada": "2026-04-15 10:50:00",
8       "usuario_name": "David",
9       "sala_nombre": "Sala Deportes"
10    },
11    {
12      "id_usuario": 3,
13      "id_sala": 2
```

GET /usuario-salas/{idSala}



127.0.0.1:8000/users X 127.0.0.1:8000/api/catego...

GET Send

Query Headers³ Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Status: 200 OK Size: 184 Bytes Time: 191 ms

Response Headers⁹ Cookies Results Docs

```
1 [
2   {
3     "id_usuario": 1,
4     "id_sala": 2,
5     "fecha_entrada": "2026-04-15 10:50:00",
6     "usuario_name": "David"
7   },
8   {
9     "id_usuario": 3,
10    "id_sala": 2,
11    "fecha_entrada": "2026-04-15 10:55:00",
12    "usuario_name": "David"
13  }
```

8 Conclusions

La conclusión final del proyecto es que ha sido una experiencia interesante, aunque en algunos momentos también se nos ha hecho un poco pesado. Al principio nos costó avanzar porque era una forma de trabajar diferente a la que estábamos acostumbrados, tanto a nivel de código como de organización en grupo. Esto hizo que al inicio estuviéramos algo desorganizados, pero poco a poco fuimos corrigiendo y conseguimos trabajar de forma más fluida entre los dos.

Durante el desarrollo, ambos hemos participado tanto en el backend como en el frontend, lo cual ha sido positivo para aprender de forma más completa. Empezamos construyendo la base del proyecto, es decir, la base de datos y las APIs, y después pasamos al diseño de la web. Esta parte nos llevó bastante tiempo porque quisimos cuidar los detalles, y a medida que diseñábamos también íbamos desarrollando los componibles de cada vista. Finalmente, implementamos la funcionalidad del juego.

En general, ha sido un proyecto en el que hemos aprendido bastante, sobre todo en el uso de Laravel, la creación de APIs y el trabajo con Vue. Aunque es cierto que al final fuimos algo justos de tiempo, hemos conseguido sacar el proyecto adelante y cumplir con los objetivos principales.

9 Referències

- [1] <https://laravel.com/>
- [2] <https://vuejs.org/>
- [3] <https://axios-http.com/>
- [4] <https://getbootstrap.com/>
- [5] <https://getbootstrap.com>
- [6] <https://dribbble.com/shots/26546011-game-stream-landing-page-web-ui-design>
- [7] <https://developer.mozilla.org/es/docs/Web/HTTP>
- [8]